

Suporte do sistema operacional

8.1 Visão geral do sistema operacional

- Objetivos e funções do sistema operacional
- Tipos de sistemas operacionais

8.2 Escalonamento

- Escalonamento de longo prazo
- Escalonamento de médio prazo
- Escalonamento a curto prazo

8.3 Gerenciamento de memória

- Troca de processos na memória — *Swapping*
- Particionamento
- Paginação
- Memória virtual
- *Translation lookaside buffer*
- Segmentação

8.4 Gerenciamento de memória no Pentium

- Espaços de endereços
- Segmentação
- Paginação

8.5 Gerenciamento de memória no ARM

- Organização do sistema de memória
- Tradução de endereço da memória virtual
- Formatos de gerenciamento de memória
- Controle de acesso

8.6 Leitura recomendada e sites Web

PRINCIPAIS PONTOS

- O sistema operacional (SO) é o software que controla a execução de programas em um processador e que gerencia os recursos deste. Diversas funções realizadas pelo sistema operacional, incluindo o escalonamento de processo e o gerenciamento de memória, só podem ser realizadas de modo eficiente e rápido se o hardware do processador incluir capacidades para dar suporte ao SO. Praticamente todos os processadores incluem essas capacidades de uma forma ou de outra, incluindo o hardware de gerenciamento de memória virtual e o hardware de gerenciamento de processos. O hardware inclui registradores e buffers de uso especial, além de circuitos para realizar tarefas básicas de gerenciamento de recursos.

- Uma das funções mais importantes do SO é o escalonamento de processos, ou tarefas. O SO determina qual processo deverá ser rodado a qualquer momento. Normalmente, o hardware interrompe um processo em execução de tempos em tempos para permitir que o SO tome uma nova decisão de escalonamento de modo a compartilhar o tempo do processador de modo imparcial entre diversos processos.

- Outra função importante do SO é o gerenciamento de memória. A maioria dos sistemas operacionais contemporâneos inclui uma capacidade de memória virtual, que tem dois benefícios: (1) um processo pode ser executado na memória principal sem que todas as instruções e dados para esse programa estejam presentes na memória principal de uma só vez e (2) o espaço de memória total disponível para um programa pode ultrapassar a memória principal real no sistema. Embora o gerenciamento

de memória seja realizado no software, o SO conta com o suporte do hardware no processador, incluindo o hardware de paginação e segmentação.

Embora o foco deste texto seja o hardware do computador, existe uma área do software que precisa ser mostrada: o SO do computador. O SO é um programa que gerencia os recursos do computador, oferece serviços para os programadores e distribui a execução de outros programas. Algum conhecimento dos sistemas operacionais é essencial para apreciar os mecanismos pelos quais a CPU controla o sistema de computação. Em particular, as explicações do efeito das interrupções e do gerenciamento da hierarquia de memória são mais bem explicadas neste contexto.

O capítulo começa com uma visão geral e uma breve história dos sistemas operacionais. O núcleo do capítulo examina duas funções do SO que são mais relevantes ao estudo da organização e arquitetura do computador: escalonamento e gerenciamento de memória.

8.1 Visão geral do sistema operacional

Objetivos e funções do sistema operacional

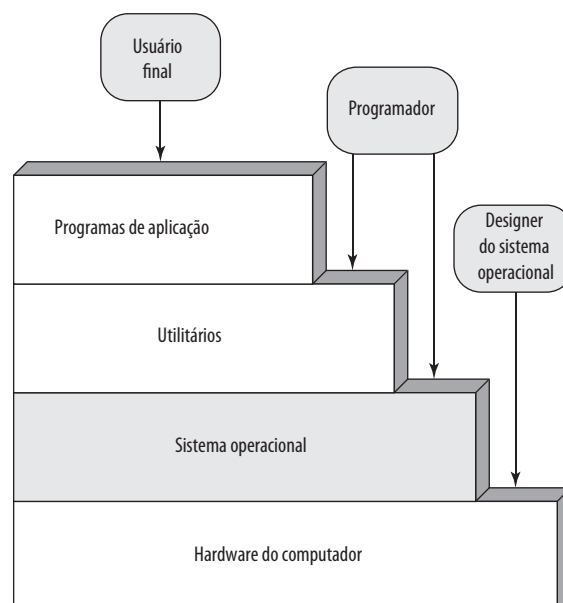
Um SO é um programa que controla a execução dos programas aplicativos e atua como uma interface entre o usuário e o hardware do computador. Ele pode ser imaginado como tendo dois objetivos:

- **Conveniência:** um SO torna um computador mais conveniente para uso.
- **Eficiência:** um SO permite que os recursos do sistema de computação sejam usados de uma maneira eficiente.

Vamos examinar esses dois aspectos do SO um de cada vez.

O SISTEMA OPERACIONAL COMO UMA INTERFACE USUÁRIO/COMPUTADOR O hardware e o software utilizados para oferecer aplicações a um usuário podem ser vistos em um padrão em camadas, ou hierárquico, conforme representado na Figura 8.1. O usuário dessas aplicações, o usuário final, geralmente não se preocupa com a arquitetura do computador. Assim, o usuário final vê um sistema de computação em termos de uma aplicação. Essa aplicação pode ser expressa em uma linguagem de programação, e é desenvolvida por um programador de aplicação. Desenvolver um programa aplicativo com um conjunto de instruções do processador, que é completamente responsável por controlar o hardware do computador, é uma tarefa muito complexa. Para facilitar essa tarefa, existe um conjunto de programas de sistemas, alguns chamados de **utilitários**. Estes implementam funções usadas com frequência, que auxiliam na criação do programa, no gerenciamento de arquivos e no controle de dispositivos de E/S. Um programador utiliza essas facilidades desenvolvendo uma aplicação, e a aplicação, enquanto está sendo executada, chama os utilitários para realizar certas funções. O programa mais importante do sistema é o SO. O SO esconde os detalhes do hardware do programador e lhe oferece uma interface conveniente para usar o sistema. Ele atua como um mediador, tornando mais fácil para o programador e os programas aplicativos acessarem e utilizarem essas facilidades e serviços.

Figura 8.1 Camadas e visões de um sistema de computação



Resumindo, o SO normalmente oferece serviços nas seguintes áreas:

- **Criação de programas:** o SO oferece diversas facilidades e serviços, como editores e depuradores, para auxiliar o programador na criação de programas. Normalmente, esses serviços são programas utilitários que não fazem realmente parte do SO, mas são acessíveis por meio dele.
- **Execução do programa:** diversas tarefas precisam ser realizadas para executar um programa. As instruções e os dados precisam ser carregados para a memória principal, os dispositivos de E/S e os arquivos precisam ser inicializados, e outros recursos precisam ser preparados. O SO trata de tudo isso para o usuário.
- **Acesso aos dispositivos de E/S:** cada dispositivo de E/S exige seu próprio conjunto específico de instruções ou sinais de controle para a operação. O SO cuida dos detalhes, de modo que o programador pode pensar em termos de simples leituras e escritas.
- **Acesso controlado aos arquivos:** no caso dos arquivos, o controle precisa incluir um conhecimento não apenas da natureza do dispositivo de E/S (unidade de disco, unidade de fita), mas também do formato de arquivo no meio de armazenamento. Novamente, o SO se preocupa com os detalhes. Além disso, no caso de um sistema com múltiplos usuários simultâneos, o SO pode oferecer mecanismos de proteção para controlar o acesso aos arquivos.
- **Acesso ao sistema:** no caso de um sistema compartilhado ou público, o SO controla o acesso ao sistema como um todo e a seus recursos específicos. A função de acesso precisa oferecer proteção de recursos e dados de usuários não autorizados, e precisa resolver conflitos para disputa de recurso.
- **Deteção e resposta a erros:** uma grande variedade de erros pode ocorrer enquanto um sistema de computação está sendo operado. Estes incluem erros de hardware internos e externos, como um erro de memória ou uma falha ou defeito de dispositivo; e diversos erros de software, como estouro aritmético, tentativa de acessar um local proibido da memória e incapacidade do SO em conceder a solicitação de uma aplicação. Em cada caso, o SO precisa tomar uma medida que encerre a condição de erro com o mínimo de impacto sobre as aplicações em execução. A resposta pode variar desde encerrar o programa que causou o erro, até tentar a operação novamente ou apenas informar o erro à aplicação.
- **Contabilidade:** um bom SO coleta estatísticas de uso para diversos recursos e monitora os parâmetros de desempenho, como o tempo de resposta. Em qualquer sistema, essa informação é útil na antecipação da necessidade de melhorias futuras e no ajuste do sistema para melhorar o desempenho. Em um sistema multiusuário, a informação pode ser usada para fins de cobrança.

O SISTEMA OPERACIONAL COMO GERENCIADOR DE RECURSOS Um computador é um conjunto de recursos para o movimento, o armazenamento e o processamento de dados e para o controle dessas funções. O SO é responsável por gerenciar esses recursos.

Podemos dizer que o SO controla o movimento, o armazenamento e o processamento de dados? Por um ponto de vista, a resposta é sim. Gerenciando os recursos do computador, o SO está no controle das funções básicas da máquina, mas esse controle é exercido de uma forma curiosa. Normalmente, pensamos em um mecanismo de controle como algo externo àquele que é controlado ou, pelo menos, como algo que é uma parte distinta e separada daquilo que é controlado. (Por exemplo, um sistema de aquecimento residencial é controlado por um termostato, que é completamente distinto do aparelho de geração e distribuição de calor.) Isso não acontece com o SO, que, como um mecanismo de controle, é incomum em dois aspectos:

- O SO funciona da mesma maneira que o software comum do computador; ou seja, ele é um programa executado pelo processador.
- O SO frequentemente abre mão do controle e precisa depender do processador para permitir que ele adquira o controle.

O SO, de fato, não é nada mais do que um programa do computador. Assim como outros programas, ele oferece instruções para o processador. A principal diferença está na intenção do programa. O SO direciona o processador no uso dos outros recursos do sistema e na sincronização de sua execução dos outros programas. Mas, para que o processador faça alguma dessas coisas, ele precisa deixar de executar o programa do SO e executar outros programas. Assim, o SO abre mão do controle para o processador realizar algum trabalho “útil” e depois retoma o controle por tempo suficiente para preparar o processador para realizar o próximo trabalho. Os mecanismos envolvidos em tudo isso deverão se tornar claros à medida que prosseguimos no capítulo.

A Figura 8.2 sugere os principais recursos que são gerenciados pelo SO. Uma parte do SO está na memória principal. Isso inclui o **kernel**, ou **núcleo**, que contém as funções mais utilizadas no SO e, em determinado momento, outras partes do SO em uso atualmente. O restante da memória principal contém programas e dados do usuário. A alocação desse recurso (a memória principal) é controlada juntamente pelo SO e pelo hardware de gerenciamento de memória no processador, conforme veremos. O SO decide quando o dispositivo de E/S pode ser usado por um programa em execução, e controla o acesso e o uso dos arquivos. O próprio processador é um recurso, e o SO precisa determinar quanto tempo do processador deve ser dedicado à execução de determinado programa do usuário. No caso de um sistema com múltiplos processadores, essa decisão precisa abranger todos os processadores.



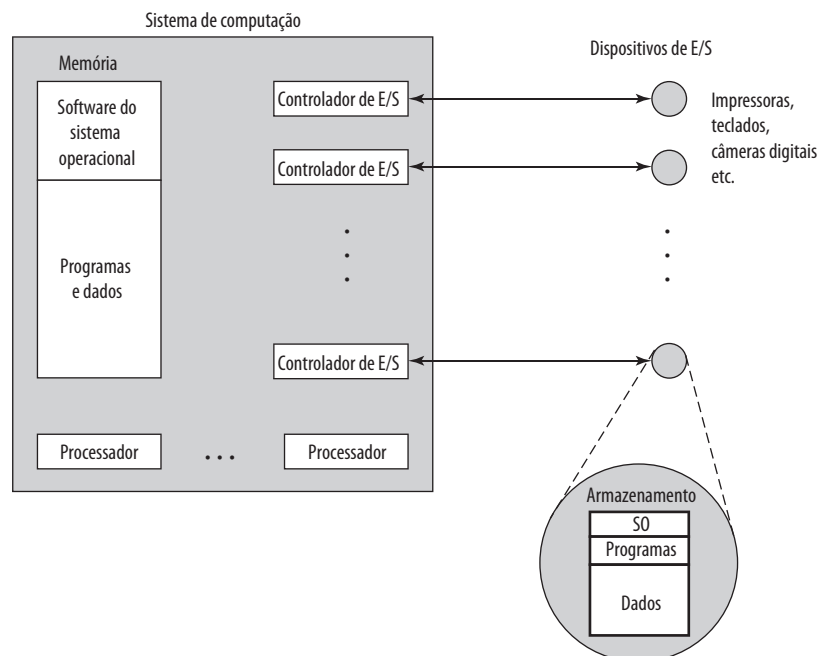
Tipos de sistemas operacionais

Certas características básicas servem para diferenciar diversos tipos de sistemas operacionais. As características se espalham por duas dimensões independentes. A primeira dimensão especifica se o sistema é em lote ou interativo. Em um sistema **interativo**, o usuário/programador interage diretamente com o computador, normalmente por meio de um terminal com teclado/vídeo, para solicitar a execução de um job (tarefa) ou realizar uma transação. Além do mais, dependendo da natureza da operação, o usuário pode se comunicar com o computador durante a execução da tarefa. Um sistema em **lote** (ou *batch*) é o oposto do interativo. O programa do usuário é mantido junto com programas de outros usuários e submetido por um operador de computador. Depois que o programa termina, os resultados são impressos para o usuário. Os sistemas puramente em lote são raros hoje. Porém, será útil para a descrição dos sistemas operacionais contemporâneos examinar os sistemas em lote rapidamente.

Uma dimensão independente especifica se o sistema emprega **multiprogramação** ou não. Com a multiprogramação, tenta-se manter o processador o mais ocupado possível, fazendo com que ele trabalhe em mais de um programa de cada vez. Vários programas são carregados na memória e o processador alterna rapidamente entre eles. A alternativa é um sistema de **uniprogramação**, que trabalha apenas com um programa de cada vez.

SISTEMAS ANTIGOS Com os computadores mais antigos, desde o final da década de 1940 até meados dos anos 1950, o programador interagiu diretamente com o hardware do computador; não havia um SO. Esses processadores eram executados a partir de um console, consistindo em lâmpadas, chaves de duas posições, alguma

Figura 8.2 O sistema operacional como gerenciador de recursos



forma de dispositivo de entrada e uma impressora. Os programas no código do processador eram carregados por meio de um dispositivo de entrada (por exemplo, uma leitora de cartões). Se um erro interrompesse o programa, a condição era indicada pelas lâmpadas. O programador poderia prosseguir para examinar os registradores e a memória principal, para determinar a causa do erro. Se o programa prosseguisse para um término normal, a saída aparecia na impressora.

Esses primeiros sistemas apresentavam dois problemas principais:

- **Escalonamento:** a maioria das instalações usava uma folha de registro para reservar o tempo do processador. Normalmente, um usuário poderia reservar um período de tempo em múltiplos de uma hora ou mais. Um usuário poderia reservar por uma hora e terminar em 45 minutos; isso resultaria em um tempo de computador ocioso desperdiçado. Por outro lado, o usuário poderia encontrar problemas, não terminar no tempo alocado e ser forçado a terminar antes de resolver o problema.
- **Tempo de preparação:** um único programa, chamado de **job**, poderia consistir em carregar o compilador mais o programa na linguagem de alto nível (programa fonte) na memória, salvar o programa compilado (programa objeto) e depois carregar e link-editar o programa objeto e as funções comuns. Cada uma dessas etapas poderia envolver a montagem ou desmontagem de fitas, ou preparação de pacotes de cartões. Se houvesse um erro, o “pobre” usuário normalmente tinha que voltar ao início da sequência de preparação. Assim, um tempo considerável era gasto apenas na preparação do programa para ser executado.

Esse modo de operação poderia ser chamado de processamento serial, refletindo o fato de que os usuários têm acesso ao computador em série. Com o passar do tempo, diversas ferramentas de software do sistema foram desenvolvidas para tentar tornar o processamento serial mais eficiente. Estas incluem bibliotecas de funções comuns, link-editores, carregadores, depuradores e rotinas de driver de E/S, que estavam disponíveis como software comum para todos os usuários.

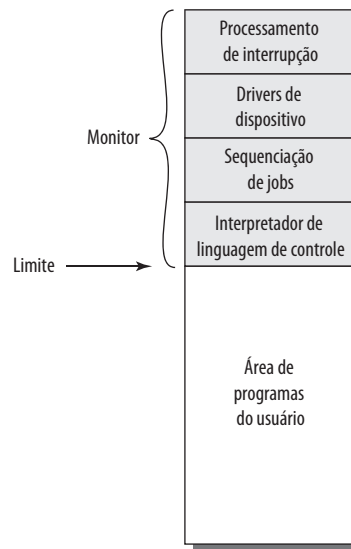
SISTEMAS EM LOTE SIMPLES Os primeiros processadores eram muito caros e, portanto, era importante maximizar a utilização deles. O tempo desperdiçado devido ao tempo de escalonamento e preparação era inaceitável.

Para melhorar a utilização, foram desenvolvidos sistemas operacionais em lote simples. Com esse tipo de sistema, também chamado de **monitor**, o usuário não tem mais acesso direto ao processador. Em vez disso, ele submete o job nos cartões ou em fita a um operador de computador, que dispõe os jobs sequencialmente e coloca o lote inteiro em um dispositivo de entrada, para uso pelo monitor.

Para entender como esse esquema funciona, vamos examiná-lo sob dois pontos de vista: o do monitor e o do processador. Pelo ponto de vista do monitor, este controla a sequência de eventos. Para que isso funcione dessa forma, grande parte do monitor precisa sempre estar na memória principal e disponível para execução (Figura 8.3). Essa parte é conhecida como **monitor residente**. O restante do monitor consiste em utilitários e funções comuns que são carregadas como sub-rotinas para o programa do usuário no início de qualquer job que as requeira. O monitor lê os jobs um de cada vez pelo dispositivo de entrada (normalmente, uma leitora de cartões ou uma unidade de fita magnética). Enquanto é lido, o job atual é colocado na área do programa do usuário e o controle é passado para esse job. Quando o job termina, ele retorna o controle ao monitor, que imediatamente lê o job seguinte. Os resultados de cada job são impressos para entrega ao usuário.

Agora, considere essa sequência do ponto de vista do processador. Em um certo ponto no tempo, ele está executando instruções a partir da parte da memória principal que contém o monitor. Essas instruções fazem com que o próximo job seja lido para outra parte da memória principal. Quando um job tiver sido lido, o processador encontrará no monitor uma instrução de desvio que instrui o processador a continuar a execução no início do programa do usuário. O processador, então, executará a instrução no programa do usuário até encontrar um final ou uma condição de erro. Qualquer um desses eventos faz com que o processador busque sua próxima instrução no programa monitor. Assim, a frase “o controle é passado a um job” simplesmente significa que o processador agora está buscando e executando instruções em um programa do usuário, e “o controle retorna ao monitor” significa que o processador agora está buscando e executando instruções do programa monitor.

Deve ter ficado claro que o monitor resolve o problema do escalonamento. Um lote de jobs é enfileirado e os jobs são executados o mais rapidamente possível, sem um tempo ocioso intercalado.

Figura 8.3 Layout de memória para um monitor residente

E o tempo de preparação? O monitor trata disso também. Com cada job, as instruções são incluídas em uma linguagem de controle de job (JCL, do inglês *job control language*). Esse é um tipo especial de linguagem de programação usada para fornecer instruções ao monitor. Um exemplo simples é o de um usuário submetendo um programa escrito em FORTRAN mais alguns dados a serem usados pelo programa. Cada instrução em FORTRAN e cada item de dados está em um cartão perfurado separado, ou em um registro separado na fita. Além do FORTRAN e das linhas de dados, o job inclui instruções de controle de job, que são indicadas com o início “\$”. O formato geral do job se parece com o seguinte:

```

$JOB
$FTN
...    } Instruções em FORTRAN
$LOAD
$RUN
...    } Dados
$END

```

Para executar esse job, o monitor lê a linha \$FTN e carrega o compilador apropriado do seu armazenamento em massa (normalmente, fita). O compilador traduz o programa do usuário para um código objeto, que é armazenado na memória ou no armazenamento em massa. Se for armazenado na memória, a operação é chamada de “compilar, carregar e executar”. Se for armazenado em fita, então a instrução \$LOAD é necessária. Essa instrução é lida pelo monitor, que readquire o controle após a operação de compilação. O monitor chama o *loader* (carregador), que carrega o programa objeto na memória no lugar do compilador e transfere o controle para ele. Dessa maneira, um grande segmento da memória principal pode ser compartilhado entre diferentes subsistemas, embora somente um subsistema possa estar residente e executando de cada vez.

Vemos que o monitor, ou SO em lote, é simplesmente um programa de computador. Ele conta com a capacidade do processador de buscar instruções de várias partes da memória principal, a fim de obter e abrir mão do controle alternadamente. Certos outros recursos do hardware também são desejáveis:

- **Proteção da memória:** enquanto o programa do usuário está sendo executado, ele não pode alterar a área da memória contendo o monitor. Se isso for tentado, o hardware do processador deverá detectar um erro e transferir o controle ao monitor. O monitor, então, abortaria o job, imprimiria uma mensagem de erro e carregaria o próximo job.

- **Temporizador:** um temporizador é usado para impedir que um único job monopolize o sistema. Ele é definido no início de cada job. Se o temporizador expirar, ocorrerá uma interrupção e o controle retornará ao monitor.
- **Instruções privilegiadas:** certas instruções são designadas como privilegiadas e podem ser executadas apenas pelo monitor. Se o processador encontrar tal instrução enquanto executa um programa do usuário, haverá uma interrupção de erro. Entre as instruções privilegiadas estão as instruções de E/S, de modo que o monitor retém o controle de todos os dispositivos de E/S. Isso impede, por exemplo, que um programa do usuário acidentalmente leia instruções de controle de job do próximo job. Se um programa do usuário quiser realizar E/S, ele terá que solicitar que o monitor realize a operação para ele. Se uma instrução privilegiada for encontrada pelo processador enquanto estiver executando um programa do usuário, o hardware do processador considerará isso como um erro e transferirá o controle ao monitor.
- **Interrupções:** os modelos antigos de computador não tinham essa capacidade. Esse recurso dá ao SO mais flexibilidade para abrir mão do controle para os programas do usuário e readquirir o controle deles.

O tempo do processador alterna entre a execução dos programas do usuário e a execução do monitor. Existem duas penalidades: alguma memória principal agora é dada para o monitor e algum tempo do processador é consumido pelo monitor. Ambas são formas de *overhead*, ou sobrecarga. Mesmo havendo esse tipo de *overhead*, o sistema em lote simples melhora a utilização do computador.

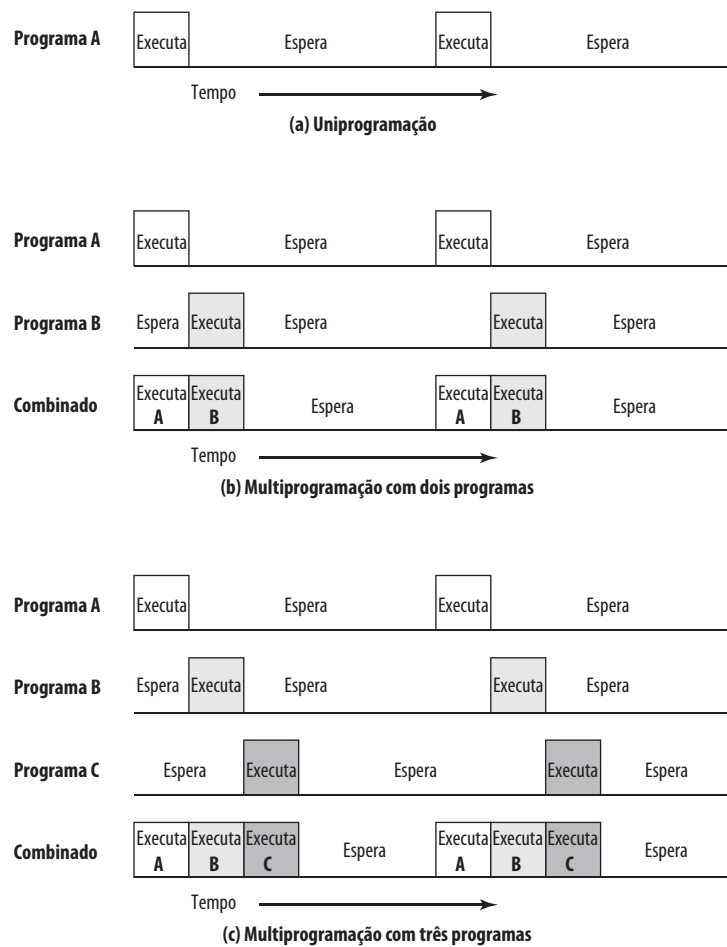
SISTEMAS EM LOTE MULTIPROGRAMADOS Mesmo com a sequenciação de job automática fornecida por um SO em lote simples, o processador constantemente fica ocioso. O problema é que os dispositivos de E/S são lentos em comparação com o processador. A Figura 8.4 detalha um cálculo representativo referente a um programa que processa um arquivo de registradores e realiza, na média, 100 instruções do processador por registro. Neste exemplo, o computador gasta mais de 96% do seu tempo esperando que os dispositivos de E/S terminem de transferir dados! A Figura 8.5a ilustra essa situação. O processador gasta um certo tempo executando, até que alcance uma instrução de E/S. Ele precisa, então, esperar até que a instrução de E/S termine para poder prosseguir.

Essa ineficiência não é necessária. Sabemos que deve haver memória suficiente para manter o SO (monitor residente) e um programa do usuário. Suponha que haja espaço para o SO e dois programas do usuário. Agora, quando um job precisar esperar pela E/S, o processador pode alternar para outro job, que provavelmente não está esperando pela E/S (Figura 8.5b). Além do mais, poderíamos expandir a memória para manter três, quatro ou mais programas e alternar entre eles (Figura 8.5c). Essa técnica é conhecida como **multiprogramação**, ou **multitarefa**.¹ Esse é o tema central dos sistemas operacionais modernos.

Figura 8.4 Exemplo de utilização do sistema

Lê um registro do arquivo	15 μ s
Executa 100 instruções	1 μ s
Grava um registro no arquivo	15 μ s
TOTAL	31 μ s
Percentual de utilização da CPU = $\frac{1}{31} = 0,032 = 3,2\%$	

¹ O termo *multitarefa*, às vezes, é reservado para indicar múltiplas tarefas dentro do mesmo programa, que podem ser tratadas simultaneamente pelo SO, ao contrário da *multiprogramação*, que se refere a múltiplos processos de múltiplos programas. Porém, é mais comum igualar os termos *multitarefa* e *multiprogramação*, como é feito na maioria dos dicionários de padrões (por exemplo, IEEE Std 100-1992, *The New IEEE Standard Dictionary of Electrical and Electronics Terms*).

Figura 8.5 Exemplo de multiprogramação

Exemplo 8.1 Este exemplo ilustra o benefício da multiprogramação. Considere um computador com 250 MBytes de memória disponível (não usada pelo SO), um disco, um terminal e uma impressora. Três programas, TAREFA1, TAREFA2 e TAREFA3, são submetidos para execução ao mesmo tempo, com os atributos listados na Tabela 8.1. Consideramos requisitos mínimos de processador para TAREFA2 e TAREFA3, e uso contínuo de disco e impressora para TAREFA3. Para um ambiente em lote simples, essas tarefas serão executadas em sequência. Assim, TAREFA1 completa em 5 minutos. TAREFA2 precisa esperar até que os 5 minutos terminem e depois termina 15 minutos após. TAREFA3 começa após 20 minutos e termina a 30 minutos do momento em que foi submetido inicialmente. A utilização média de recursos, vazão (*through put*) e tempo de resposta aparecem na coluna de uniprogramação da Tabela 8.2. A utilização dispositivo por dispositivo é ilustrada na Figura 8.6a. É evidente que existe uma subutilização bruta para todos os recursos quando calculados na média pelo período exigido de 30 minutos.

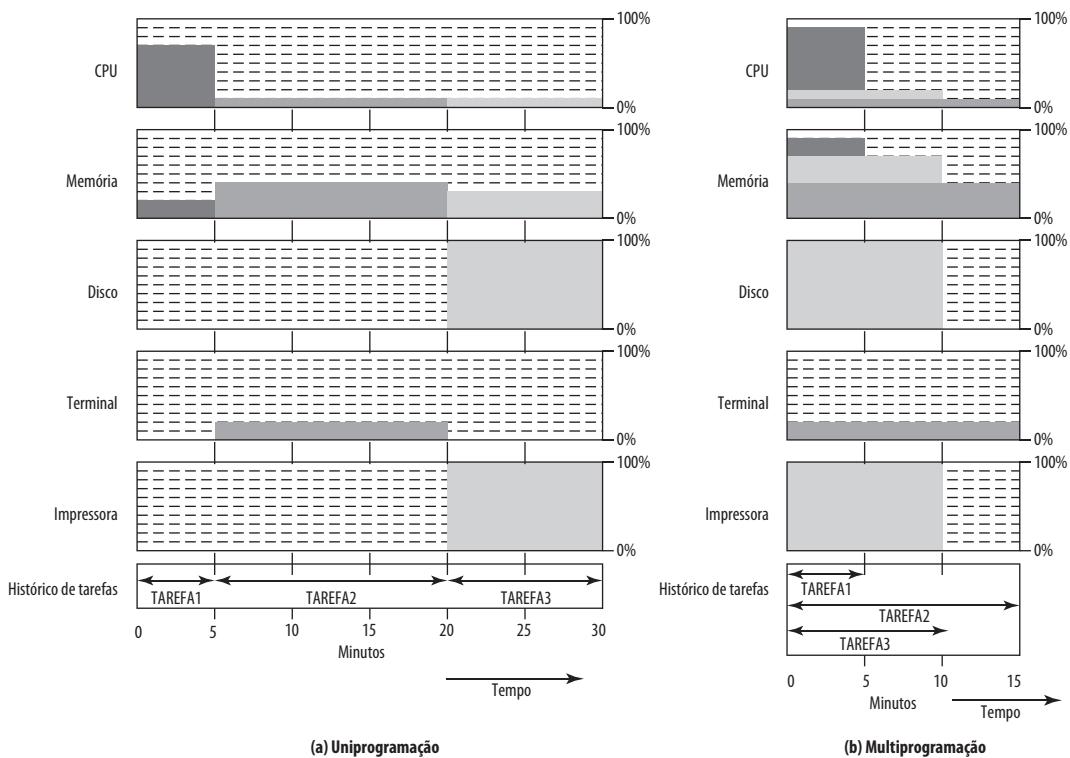
Agora, suponha que as tarefas sejam executadas simultaneamente sob um SO de multiprogramação. Como existe pouca disputa de recursos entre as tarefas, todas elas podem ser executadas em um tempo quase mínimo, enquanto coexistem com os outros no computador (supondo que TAREFA2 e TAREFA3 tenham recebido tempo de processador suficiente para manter suas operações de entrada e saída ativas). TAREFA1 exigirá 5 minutos para terminar, mas, ao final desse tempo, TAREFA2 estará 1/3 acabado, e TAREFA3 já estará pela metade. As três tarefas terão terminado dentro de 15 minutos. A melhoria é evidente quando examinamos a coluna de multiprogramação da Tabela 8.2, obtida pelo histograma mostrado na Figura 8.6b.

Tabela 8.1 Exemplo de atributos de execução de programa

	TAREFA1	TAREFA2	TAREFA3
Tipo de tarefa	Computação pesada	E/S pesada	E/S pesada
Duração	5 min	15 min	10 min
Memória exigida	50 M	100 M	80 M
Precisa de disco?	Não	Não	Sim
Precisa de terminal?	Não	Sim	Não
Precisa de impressora?	Não	Não	Sim

Tabela 8.2 Efeitos da multiprogramação sobre a utilização de recursos

	Uniprogramação	Multiprogramação
Uso de processador	20%	40%
Uso de memória	33%	67%
Uso de disco	33%	67%
Uso de impressora	33%	67%
Tempo decorrido	30 min	15 min
Taxa de <i>through put</i>	6 tarefas/h	12 tarefas/h
Tempo médio de resposta	18 min	10 min

Figura 8.6 Histogramas de utilização

Assim como em um sistema em lote simples, um sistema em lote com multiprogramação precisa contar com certos recursos de hardware do computador. O recurso adicional mais relevante, que é útil para a multiprogramação, é o hardware que dá suporte a interrupções de E/S e DMA. Com a E/S controlada por interrupção ou DMA, o processador pode emitir um comando de E/S para um job e prosseguir com a execução de outro job enquanto a E/S é executada pelo controlador de dispositivo. Quando a operação de E/S termina, o processador é interrompido e o controle é passado para uma rotina de tratamento de interrupção no SO. Este, então, passa o controle a outro job.

Os sistemas operacionais com multiprogramação são bastante sofisticados em comparação com os sistemas de único programa, ou **uniprogramação**. Para ter vários jobs prontos para execução, os jobs precisam ser mantidos na memória principal, exigindo alguma forma de **gerenciamento de memória**. Além disso, se vários jobs estiverem prontos para executar, o processador precisa decidir qual executar, o que requer algum algoritmo para escalonamento. Esses conceitos são discutidos mais adiante neste capítulo.

SISTEMAS DE TEMPO COMPARTILHADO Com o uso da multiprogramação, o processamento em lote pode ser bastante eficiente. Porém, para muitos jobs, é desejável oferecer um modo em que o usuário interaja diretamente com o computador. Na realidade, para alguns jobs, como o processamento de transações, um modo interativo é essencial.

Hoje, o requisito para uma facilidade de computação interativa pode ser, e às vezes é, atendido pelo uso de um microcomputador dedicado. Essa opção não estava disponível na década de 1960, quando quase todos os computadores eram grandes e caros. Em vez disso, foi desenvolvido o sistema de tempo compartilhado.

Assim como a multiprogramação permite que o processador trate de múltiplos jobs em lote de uma só vez, ela pode ser usada para lidar com múltiplos jobs interativos. Nesse último caso, a técnica é chamada de tempo compartilhado: o tempo do processador é compartilhado entre vários usuários. Em um sistema de tempo compartilhado, vários usuários acessam o sistema simultaneamente por meio de terminais, com o SO intercalando a execução de cada programa do usuário em um curto intervalo de tempo ou *quantum* de computação. Assim, se houver n usuários ativamente solicitando serviço de uma só vez, cada usuário só verá uma média de $1/n$ da velocidade efetiva do computador, sem contar o *overhead* do SO. Porém, dado o tempo de reação relativamente lento do ser humano, o tempo de resposta em um sistema corretamente projetado deverá ser comparável ao de um computador dedicado.

Tanto a multiprogramação em lote quanto o compartilhamento de tempo utilizam multiprogramação. As principais diferenças estão listadas na Tabela 8.3.



8.2 Escalonamento

A chave para a multiprogramação é o escalonamento. De fato, quatro tipos de escalonamento normalmente são envolvidos (Tabela 8.4) e vamos explorá-los nesta seção. Mas, antes, vamos apresentar o conceito de **processo**. Esse termo foi usado inicialmente pelos projetistas do SO MULTICS na década de 1960. Esse é um termo um pouco mais generalizado do que *job*. Muitas definições têm sido dadas para o termo *processo*, incluindo:

- Um programa em execução.
- O “espírito animado” de um programa.
- A entidade à qual um processador é atribuído.

Esse conceito deverá se tornar mais claro à medida que prosseguirmos.

Tabela 8.3 Multiprogramação em lote *versus* tempo compartilhado

	Multiprogramação em lote	Tempo compartilhado
Objetivo principal	Maximizar uso do processador	Minimizar tempo de resposta
Origem das diretivas ao sistema operacional	Comandos da JCL fornecidos com a tarefa	Comandos digitados no terminal

Tabela 8.4 Tipos de escalonamento

Escalonamento de longo prazo	A decisão de acrescentar ao pool de processos a serem executados
Escalonamento a médio prazo	A decisão de acrescentar ao número de processos que estão parcial ou totalmente na memória principal
Escalonamento de curto prazo	A decisão sobre qual processo disponível será executado pelo processador
Escalonamento de E/S	A decisão sobre qual solicitação de E/S pendente do processo será tratada por um dispositivo de E/S disponível



Escalonamento de longo prazo

Um escalonador a longo prazo determina quais programas são admitidos no sistema para processamento. Assim, ele controla o grau de multiprogramação (número de processos na memória). Uma vez admitido, um job ou programa do usuário se torna um processo e é acrescentado à fila para o escalonador de curto prazo. Em alguns sistemas, um processo recém-criado inicia em uma condição não carregado na memória (*swapped-out*), quando é acrescentado a uma fila para o escalonador a médio prazo.

Em um sistema em lote, ou para a parte de lote de um SO de uso geral, os jobs recém-submetidos são direcionados para o disco e mantidos em uma fila de lote. O escalonador de longo prazo cria processos a partir da fila, quando possível. Existem duas decisões envolvidas aqui. Primeiro, o escalonador precisa decidir se o SO pode assumir um ou mais processos adicionais. Segundo, o escalonador precisa decidir qual job ou jobs irá aceitar e transformar em processos. Os critérios usados podem incluir prioridade, tempo de execução esperado e requisitos de E/S.

Para programas interativos em um sistema de tempo compartilhado, uma requisição de processo é gerada quando um usuário tenta se conectar ao sistema. Os usuários de tempo compartilhado não são simplesmente enfileirados e mantidos esperando até que o sistema possa aceitá-los. Em vez disso, o SO aceitará todos os que chegam autorizados até que o sistema esteja saturado, usando alguma medida de saturação predefinida. Nesse ponto, uma requisição de conexão é atendida com uma mensagem indicando que o sistema está cheio e que o usuário deverá tentar novamente mais tarde.



Escalonamento de médio prazo

O escalonamento de médio prazo faz parte da função de troca de processo (*swapping*), descrita na Seção 8.3. Normalmente, a decisão de entrada no swapping é baseada na necessidade de gerenciar o grau de multiprogramação. Em um sistema que não usa memória virtual, o gerenciamento de memória também é um ponto. Assim, a decisão de processo de memória considerará os requisitos de memória dos processos que são removidos para o disco.

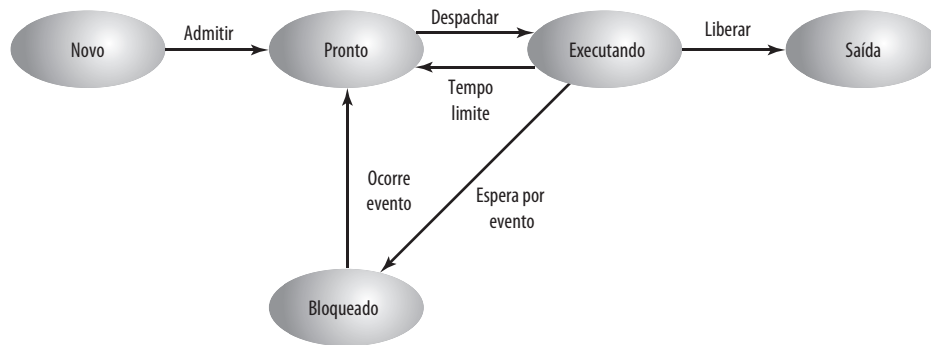


Escalonamento a curto prazo

O escalonador de longo prazo é executado com relativamente pouca frequência e toma a decisão bruta de assumir ou não um novo processo, e qual deverá assumir. O escalonador de curto prazo, também conhecido como **despachante** (*dispatcher*), é executado com frequência e toma a decisão de nível mais baixo de qual tarefa executar em seguida.

ESTADOS DE PROCESSOS Para entender a operação do escalonador de curto prazo, precisamos considerar o conceito de um **estado de processo**. Durante o tempo de vida de um processo, seu *status* mudará diversas vezes. Seu estado em determinado momento é conhecido como um *estado*. O termo *estado* é usado porque ele indica que existe uma certa informação que define o estado nesse ponto. No mínimo, existem cinco estados definidos para um processo (Figura 8.7):

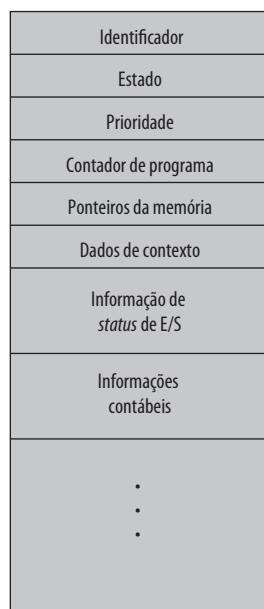
- **Novo:** o programa é admitido mas não está pronto para executar. O SO iniciará o processo, movendo-o para o estado pronto.

Figura 8.7 Modelo de processo com cinco estados

- **Pronto:** o processo está pronto para ser executado e está aguardando o acesso ao processador.
- **Em execução:** o processo está sendo executado pelo processador.
- **Suspenso:** o processo está com sua execução suspensa, aguardando por algum recurso do sistema, como a E/S.
- **Concluído:** o processo terminou e será destruído pelo SO.

Para cada processo no sistema, o SO precisa manter informações indicando o estado do processo e outras informações necessárias para a execução do processo. Para essa finalidade, cada processo é representado pelo SO por um **bloco de controle de processo** (Figura 8.8), que normalmente contém:

- **Identificador:** cada processo ativo possui um identificador exclusivo.
- **Estado:** o estado atual do processo (novo, pronto etc.).
- **Prioridade:** nível de prioridade relativo.
- **Contador de programa:** o endereço da próxima instrução no programa a ser executado.
- **Ponteiros de memória:** os locais inicial e final do processo na memória.
- **Dados de contexto:** estes são dados que estão presentes nos registradores do processador enquanto o processo está executando, e serão discutidos na Parte 3. Por enquanto, basta dizer que esses dados representam o

Figura 8.8 Bloco de controle do processo

“contexto” do processo. Os dados de contexto mais o contador de programa são salvos quando o processo sai do estado de execução. Eles são recuperados pelo processador quando ele retoma a execução do processo.

- **Informações de status de E/S:** inclui requisições de E/S pendentes, dispositivos de E/S (por exemplo, unidades de fita) atribuídos a esse processo, uma lista de arquivos atribuídos ao processo, e assim por diante.
- **Informações contábeis:** podem incluir a quantidade de tempo de processador e tempo de clock utilizado, limites de tempo, números de conta e assim por diante.

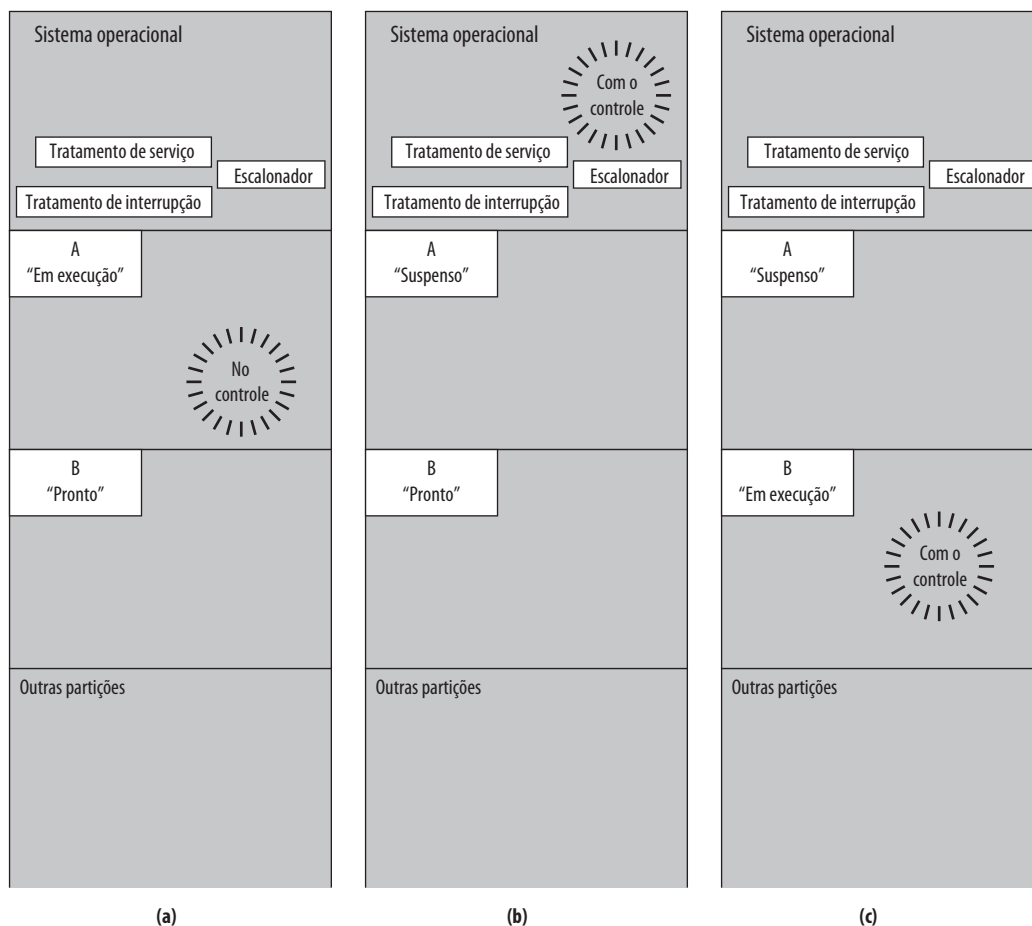
Quando o escalonador aceita um novo job ou requisição de um usuário para execução, ele cria um bloco de controle de processo em branco e coloca o processo associado no estado novo. Depois que o sistema tiver preenchido corretamente o bloco do controle de processo, o processo é transferido para o estado pronto.

TÉCNICAS DE ESCALONAMENTO Para entender como o SO gerencia o escalonamento de diversos jobs na memória, vamos começar considerando o exemplo simples na Figura 8.9. A figura mostra como a memória principal é particionada em determinado momento. O kernel do SO, naturalmente, sempre está residente. Além disso, existem diversos processos ativos, incluindo A e B, cada um recebendo a alocação de uma parte da memória.

Começamos em um ponto no tempo em que o processo A está sendo executado. O processador está executando as instruções do programa contido na partição de memória de A. Em algum outro momento, o processador deixa de executar as instruções em A e começa a executar as instruções na área do SO. Isso acontecerá por uma destas três razões:

1. O processo A emite uma chamada de serviço (por exemplo, uma requisição de E/S) ao SO. A execução de A é suspensa até que essa chamada seja satisfeita pelo SO.

Figura 8.9 Exemplo de escalonamento



2. O processo A causa uma *interrupção*, que é um sinal gerado pelo hardware ao processador. Quando esse sinal é detectado, o processador deixa de executar A e transfere a execução ao tratamento de interrupção no SO. Diversos eventos relacionados a A causarão uma interrupção. Um exemplo é um erro, como a tentativa de executar uma instrução privilegiada. Outro exemplo é um tempo limite esgotado (*timeout*); para impedir que qualquer processo monopolize o processador, cada processo só recebe a atenção do processador; por um curto período de cada vez.
3. Algum evento não relacionado ao processo A que requeira atenção causa uma interrupção. Um exemplo é o término de uma operação de E/S.

De qualquer forma, o resultado é o seguinte: o processador salva os dados de contexto atuais e o contador de programa para A no bloco de controle do processo de A, e depois começa a executar no SO. O SO pode realizar algum trabalho, como iniciar uma operação de E/S. Depois, a parte do escalonador de curto prazo do SO decide qual processo deverá ser executado em seguida. Neste exemplo, B é escolhido. O SO instrui o processador a restaurar os dados de contexto de B e prosseguir com a execução de B onde ele parou.

Este exemplo simples destaca o funcionamento básico do escalonador a curto prazo. A Figura 8.10 mostra os principais elementos do SO envolvidos na multiprogramação e escalonamento de processos. O SO recebe o controle do processador no tratamento de interrupção se ocorrer uma interrupção, e no tratamento de chamada de serviço se ocorrer uma chamada de sistema. Quando uma interrupção ou chamada de serviço for tratada, o escalonador a curto prazo será chamado para selecionar um processo para execução.

Para realizar sua tarefa, o SO mantém diversas filas. Cada fila é simplesmente uma lista de espera dos processos aguardando por algum recurso. A **fila de longo prazo** é uma lista dos jobs aguardando para usar o sistema. Quando as condições permitirem, o escalonador de alto nível alocará memória e criará um processo para um dos itens que está aguardando. A **fila de curto prazo** consiste em todos os processos no estado pronto. Qualquer um desses processos poderia usar o processador em seguida. Fica a cargo do escalonador a curto prazo escolher um. Geralmente, isso é feito com um algoritmo *round-robin*, dando a cada processo algum tempo, em forma de rodízio. Os níveis de prioridade também podem ser usados. Finalmente, existe uma **fila de E/S** para cada dispositivo de E/S. Mais de um processo pode requisitar o uso do mesmo dispositivo de E/S. Todos os processos aguardando para usar cada dispositivo são alinhados na fila desse dispositivo.

A Figura 8.11 sugere como os processos progridem pelo computador sob o controle do SO. Cada requisição de processo (job em lote, job interativo definido pelo usuário) é colocado na fila de longo prazo. Quando os recursos

Figura 8.10 Principais elementos de um sistema operacional para multiprogramação

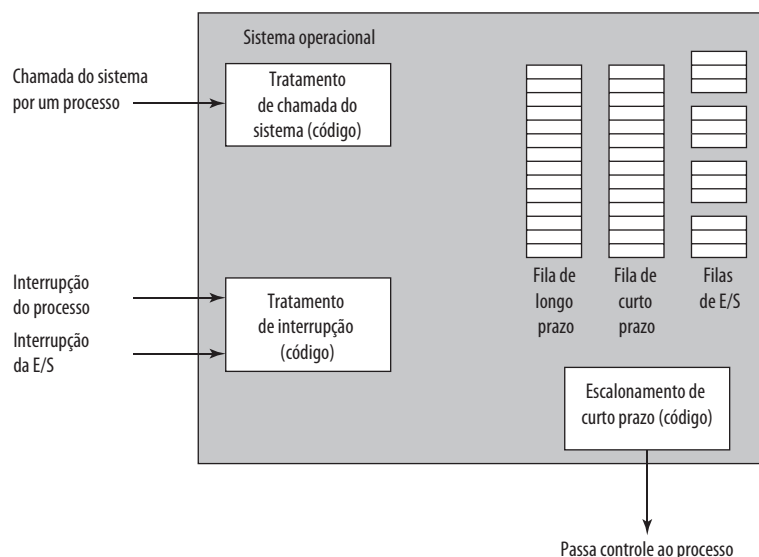
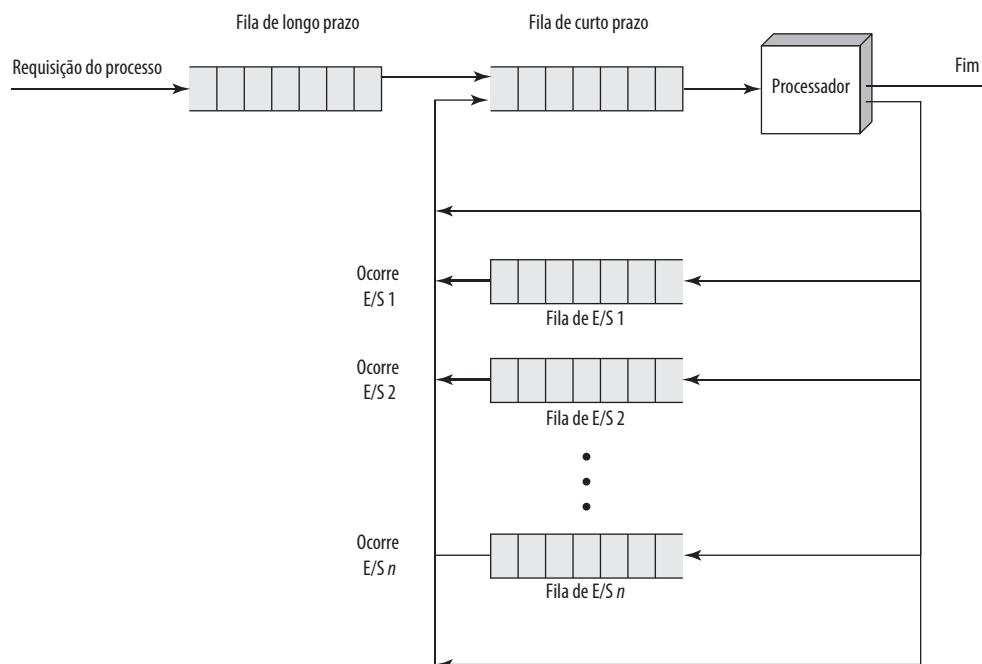


Figura 8.11 Diagrama de filas do escalonamento de processador

se tornam disponíveis, uma requisição do processo se torna um processo, que é então colocado no estado pronto e mantido na fila de curto prazo. O processador alterna entre executar instruções do SO e executar processos do usuário. Enquanto o SO está no controle, ele decide qual processo na fila de curto prazo deverá ser executado em seguida. Quando o SO termina suas tarefas imediatas, ele transfere o processador para o processo escolhido.

Como já dissemos, um processo sendo executado pode ser suspenso por diversos motivos. Se ele for suspenso porque o processo requisita E/S, então ele é colocado na fila de E/S apropriada. Se ele for suspenso porque terminou seu tempo ou porque o SO precisa atender a uma atividade urgente, então ele é passado para o estado pronto e colocado na fila a curto prazo.

Finalmente, mencionamos que o SO também gerencia as filas de E/S. Quando uma operação de E/S termina, o SO remove o referido processo atendido dessa fila de E/S e o coloca na fila de curto prazo. Depois, ele seleciona outro processo que está esperando (se houver) e sinaliza o dispositivo de E/S para que atenda a requisição desse processo.



8.3 Gerenciamento de memória

Em um sistema de uniprogramação, a memória principal é dividida em duas partes: uma parte para o SO (monitor residente) e uma parte para o programa atualmente sendo executado. Em um sistema de multiprogramação, a parte do “usuário” da memória é subdividida para acomodar diversos processos. A tarefa de subdivisão é executada dinamicamente pelo SO e é conhecida como **gerenciamento de memória**.

O gerenciamento de memória eficaz é vital em um sistema de multiprogramação. Se apenas alguns processos estiverem na memória, então, em grande parte do tempo, todos os processos estarão esperando pela E/S e o processador estará ocioso. Assim, a memória precisa ser alocada de modo eficiente para encaixar o máximo de processos possível na memória.



Troca de processos na memória – Swapping

Voltando à Figura 8.11, discutimos três tipos de filas: a fila de longo prazo de solicitações de novos processos, a fila de curto prazo de processos prontos para usar o processador e as diversas filas de E/S dos processos que não

estão prontos para usar o processador. Lembre-se de que o motivo para esse mecanismo elaborado é que as atividades de E/S são muito mais lentas do que a computação e, portanto, o processador em um sistema de uniprogramação fica ocioso na maior parte do tempo.

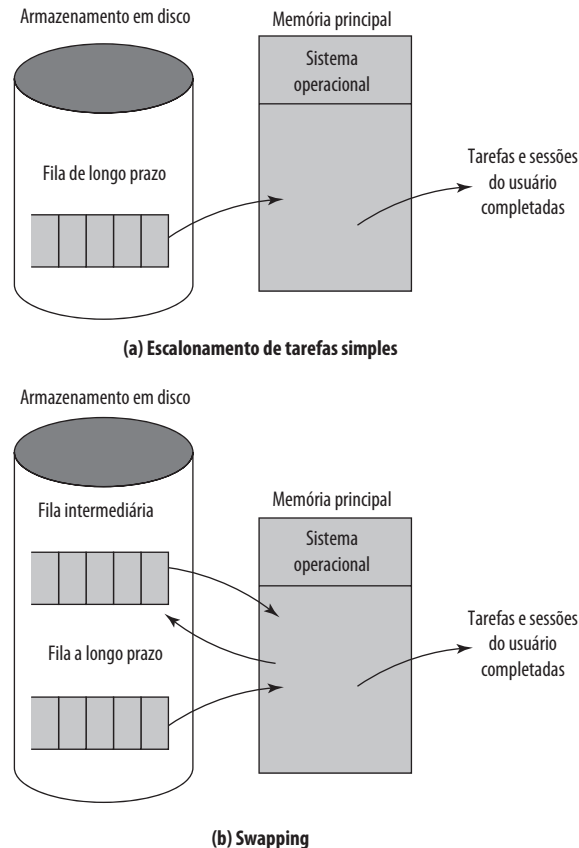
Porém, o arranjo na Figura 8.11 não resolve totalmente o problema. É verdade que, nesse caso, a memória mantém vários processos e que o processador pode passar para outro processo quando um deles estiver esperando. Mas o processador é muitas vezes mais rápido que a E/S, que será comum haver *todos* os processos na memória esperando por E/S. Assim, até mesmo com a multiprogramação, um processador poderia estar ocioso na maior parte do tempo.

O que fazer? A memória principal poderia ser expandida, e, portanto, ser capaz de acomodar mais processos. Mas existem duas falhas nessa abordagem. Primeiro, a memória principal é cara, ainda hoje. Segundo, a necessidade dos programas por memória cresceu tão rápido quanto o custo da memória caiu. Assim, uma memória maior resulta em processos maiores, e não em mais processos.

Outra solução é o *swapping*, representado na Figura 8.12. Temos uma fila de longo prazo de requisições de processos, normalmente armazenada no disco. Estes são trazidos, um por vez, à medida que houver espaço disponível. Quando os processos terminam, eles são removidos da memória principal. Agora surge a situação em que nenhum dos processos na memória estará no estado pronto (por exemplo, todos estão aguardando uma operação de E/S). Ao invés de permanecer ocioso, o processador *troca* (swaps) um desses processos de volta para o disco em uma *fila intermediária*. Essa é uma fila de processos existentes que foram temporariamente removidos da memória. O SO então traz outro processo da fila intermediária, ou então atende a requisição de um novo processo da fila de longo prazo. A execução então continua com o processo recém-chegado.

O swapping, porém, é uma operação de E/S, e, portanto, pode tornar o problema ainda pior, e não melhor. Mas, como a E/S em disco geralmente é a E/S mais rápida em um sistema (por exemplo, em comparação com a E/S de fita ou impressora), o swapping normalmente melhorará o desempenho. Um esquema mais

Figura 8.12 O uso do swapping



sofisticado, envolvendo memória virtual, melhora o desempenho em relação ao simples swapping. Isso será discutido mais adiante. Primeiro, porém, temos que preparar o terreno explicando sobre particionamento e paginação.



Particionamento

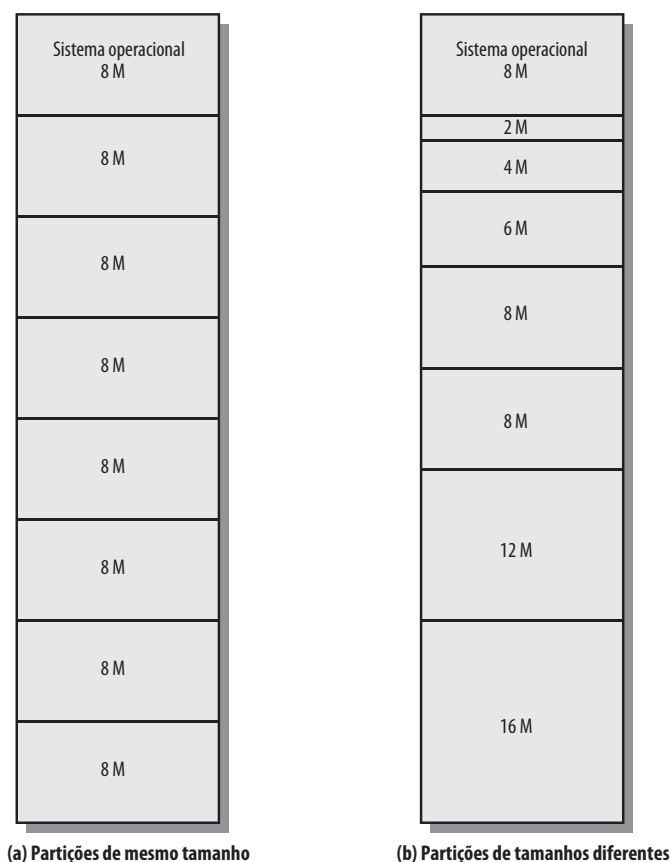
O esquema mais simples para o particionamento da memória disponível é usar *partições de tamanho fixo*, como mostramos na Figura 8.13. Observe que, embora as partições tenham tamanho fixo, elas não precisam ter o mesmo tamanho. Quando um processo é trazido para a memória, ele é colocado na menor partição possível que o poderá manter.

Mesmo com o uso de partições de tamanho fixo desiguais, haverá memória desperdiçada. Na maior parte dos casos, um processo não exigirá tanta memória quanto a fornecida pela partição. Por exemplo, um processo que requer 3 MBytes de memória seria colocado em uma partição de 4 M da Figura 8.13b, desperdiçando 1 M que poderia ser usado por outro processo.

Uma técnica mais eficiente é usar *partições de tamanho variável*. Quando um processo é levado para a memória, ele recebe exatamente a quantidade de memória exigida, e nada mais.

Exemplo 8.2 Um exemplo, usando 64 MBytes de memória principal, aparece na Figura 8.14. Inicialmente, a memória principal está vazia, exceto pelo SO (a). Os primeiros três processos são carregados nela, começando onde o SO termina e ocupando apenas o espaço suficiente para cada processo (b, c, d). Isso deixa um “buraco” ao final da memória, que é muito pequeno para um quarto processo. Em algum ponto, nenhum dos processos na memória

Figura 8.13 Exemplo de particionamento fixo de uma memória de 64 MBytes



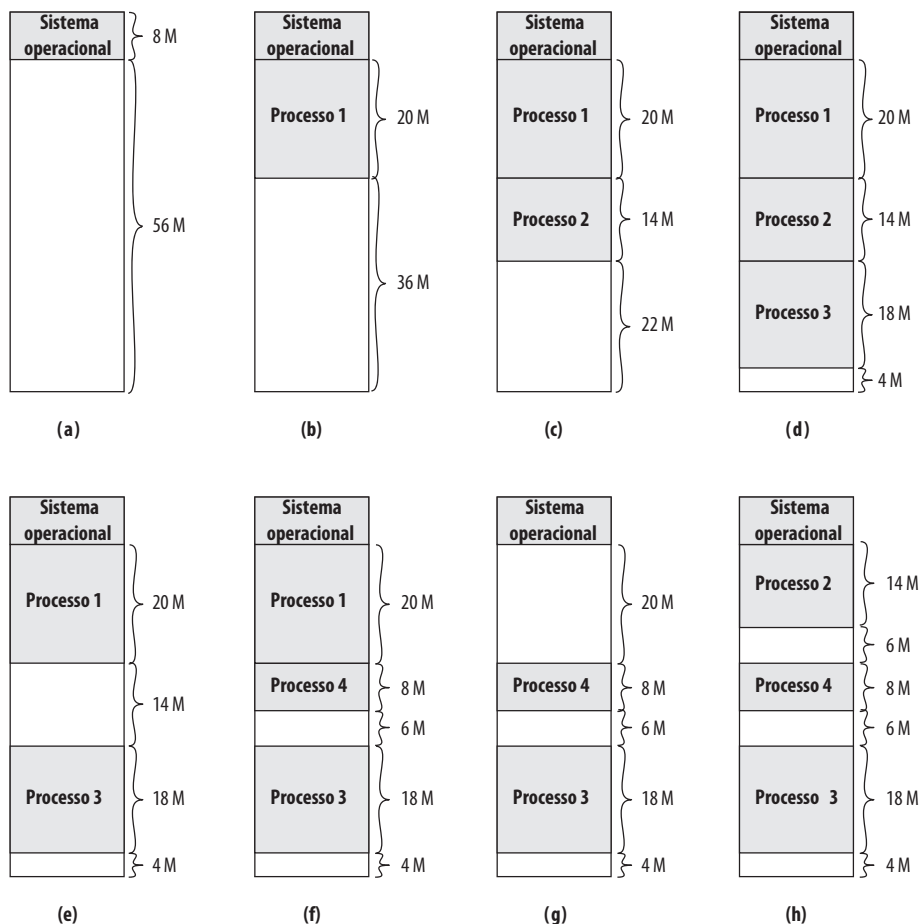
está pronto. O SO retira o processo 2 (e), o que deixa espaço suficiente para carregar um novo processo, o processo 4 (f). Como o processo 4 é menor que o processo 2, outro buraco pequeno é criado. Mais tarde, chega um ponto em que nenhum dos processos na memória principal está pronto, mas o processo 2, no estado Pronto-Suspenso, está disponível. Como existe espaço suficiente na memória para o processo 2, o SO retira o processo 1 (g) e devolve o processo 2 para a memória (h).

Como este exemplo mostra, esse método começa bem, mas, por fim, leva a uma situação em que existem muitos buracos pequenos na memória. Com o passar do tempo, a memória torna-se cada vez mais fragmentada, e sua utilização declina. Uma técnica para contornar esse problema é a **compactação**: de vez em quando, o SO desloca os processos na memória para colocar toda a memória livre junta em um só bloco. Esse é um procedimento demorado, desperdiçando o tempo do processador.

Antes de considerarmos algumas maneiras de lidar com as desvantagens do particionamento, temos que acertar um ponto. Considere a Figura 8.14; deve ficar óbvio que um processo provavelmente não será carregado para o mesmo local na memória principal a cada vez que for levado para lá. Além do mais, se houver compactação, um processo pode ser deslocado enquanto estiver na memória principal. Um processo na memória consiste em instruções mais dados. As instruções terão endereços para locais da memória de dois tipos:

- Endereços de itens de dados.
- Endereços de instruções, usados para instruções de desvio.

Figura 8.14 O efeito do particionamento dinâmico



Mas esses endereços não são fixos. Eles mudarão toda vez que um processo for levado para a memória. Para resolver esse problema, existe uma distinção entre endereços lógicos e endereços fixos. Um **endereço lógico** é expresso como um local relativo ao início do programa. As instruções no programa contêm apenas endereços lógicos. Um **endereço físico** é um local real na memória principal. Quando o processador executa um processo, ele automaticamente converte do endereço lógico para o físico, somando o local inicial atual do processo, chamado de **endereço de base**, a cada endereço lógico. Esse é outro exemplo de um recurso de hardware do processador projetado para atender a um requisito do SO. A natureza exata desse recurso de hardware depende da estratégia de gerenciamento de memória em uso. Veremos vários exemplos mais adiante neste capítulo.



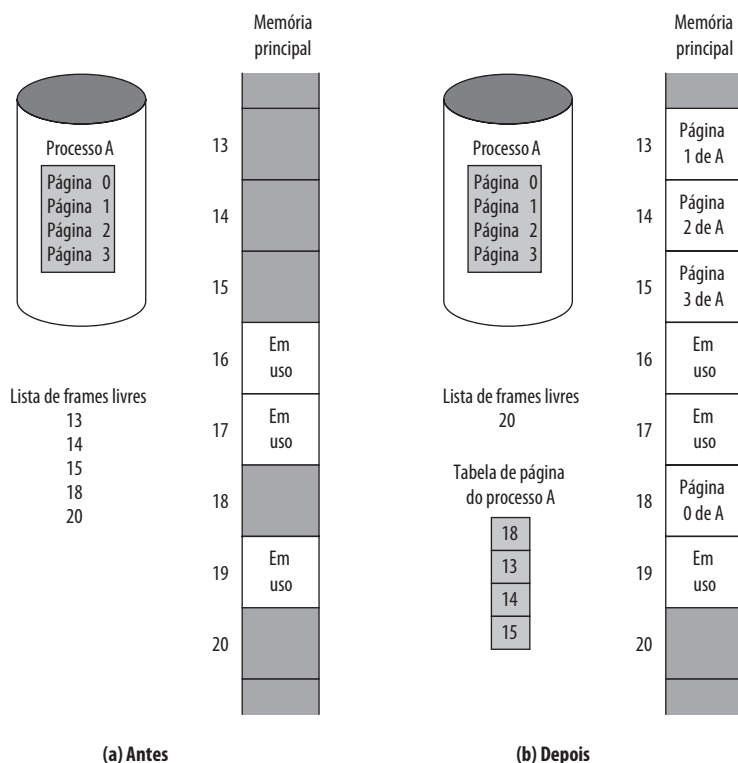
Paginação

Partições desiguais de tamanho fixo e de tamanho variável são ineficazes no uso da memória. Suponha, porém, que a memória seja particionada em pedaços iguais de tamanho fixo que sejam relativamente pequenos, e que cada processo também seja dividido em pequenos pedaços de algum tamanho fixo pequeno. Então, os pedaços de um programa, conhecidos como **páginas**, poderiam ser atribuídos aos pedaços disponíveis de memória, conhecidos como **frames**, ou frames de página. No máximo, então, o espaço desperdiçado na memória para esse processo é uma fração da última página.

A Figura 8.15 mostra um exemplo do uso de páginas e frames. Em determinado momento no tempo, alguns dos frames na memória estão em uso e alguns estão livres. A lista dos frames livres é mantida pelo SO. O processo A, armazenado no disco, consiste em quatro páginas. Quando é o momento de carregar esse processo, o SO encontra quatro frames livres e carrega as quatro páginas do processo A nos quatro frames.

Agora suponha, como neste exemplo, que não haja frames contíguos sem uso suficientes para manter o processo. Isso impede o SO de carregar A? A resposta é não, porque podemos, mais uma vez, usar o conceito de endereço lógico. Um endereço de base simples não será mais suficiente. Em vez disso, o SO mantém uma **tabela de página** para cada processo. A tabela de página mostra o local para cada página no processo. Dentro

Figura 8.15 Alocação de frames livres



do programa, cada endereço lógico consiste em um número de página e um endereço relativo dentro da página. Lembre-se de que, no caso do particionamento simples, um endereço lógico é o local de uma palavra em relação ao início do programa; o processador traduz isso para um endereço físico. Com a paginação, a tradução de endereço de lógico para físico ainda é feita pelo hardware do processador. O processador precisa saber como acessar a tabela de página do processo atual. Recebendo um endereço lógico (número de página, endereço relativo), o processador usa a tabela de página para produzir um endereço físico (número de frame, endereço relativo). Um exemplo aparece na Figura 8.16.

Essa técnica soluciona os problemas que levantamos anteriormente. A memória principal é dividida em muitos frames pequenos de mesmo tamanho. Cada processo é dividido em páginas com o tamanho do frame: processos menores exigem menos páginas, processos maiores exigem mais. Quando um processo é trazido, suas páginas são carregadas nos frames disponíveis, e uma tabela de página é montada.



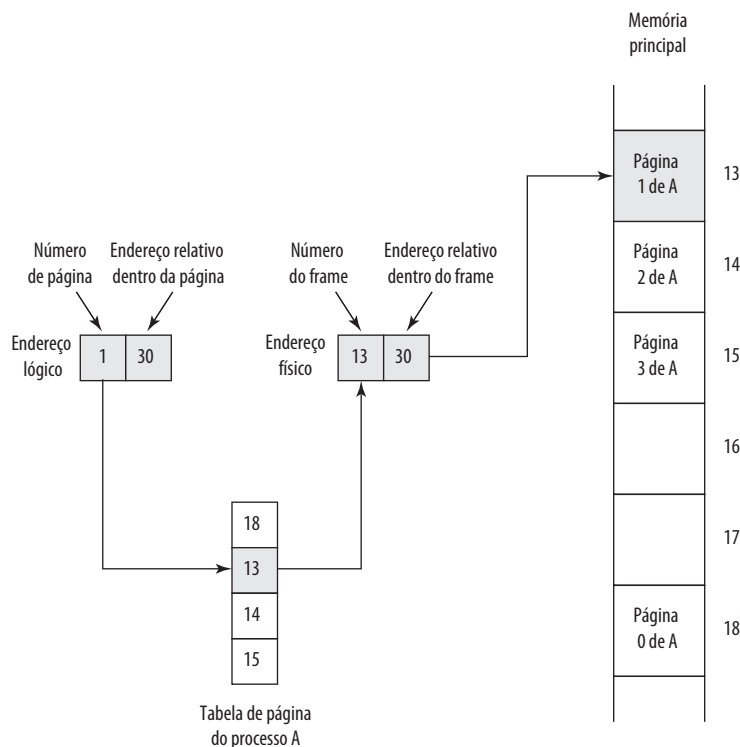
Memória virtual

PAGINAÇÃO POR DEMANDA Com o uso da paginação, sistemas de multiprogramação verdadeiramente eficazes entraram em cena. Além do mais, a simples tática de dividir um processo em páginas levou ao desenvolvimento de outro conceito importante: a memória virtual.

Para entender a memória virtual, temos que acrescentar uma melhoria ao esquema de paginação que discutimos. Essa melhoria é a **paginação por demanda**, que simplesmente significa que cada página de um processo é trazida apenas quando necessária, ou seja, por demanda.

Considere um processo grande, consistindo em um programa longo mais uma série de arrays de dados. Por qualquer período, a execução pode ser confiada a uma pequena seção do programa (por exemplo, uma sub-rotina) e talvez apenas um ou dois arrays de dados estão sendo usados. Esse é o princípio da localidade, que apresentamos no Apêndice 4A. Logicamente, seria um desperdício carregar dezenas de páginas para esse processo quando apenas algumas páginas serão usadas antes que o programa seja suspenso. Podemos

Figura 8.16 Endereços lógicos e físicos



fazer um uso melhor da memória carregando apenas algumas páginas. Depois, se o programa desviar para uma instrução em uma página fora da memória principal, ou se o programa referenciar dados em uma página que não está na memória, uma interrupção de **falta de página (page fault)** ocorrerá. Isso diz ao SO para trazer a página desejada.

Assim, a qualquer momento, apenas algumas páginas de determinado processo estão na memória, e, portanto, mais processos podem ser mantidos na memória. Além do mais, o tempo é reduzido, pois as páginas não usadas não entram e saem da memória. Porém, o SO precisa ser inteligente sobre como gerenciar esse esquema. Quando ele traz uma página, precisa retirar outra; isso é conhecido como **substituição de página**. Se ele retirar uma página imediatamente antes que ela precise ser usada, então ele simplesmente terá que obter essa página novamente, quase que imediatamente. Muita atividade desse tipo leva a uma condição conhecida como **thrashing**: o processador gasta a maior parte do seu tempo trocando páginas, ao invés de executando instruções. Impedir o thrashing foi uma área de pesquisa importante na década de 1970, e levou a uma série de algoritmos complexos, porém eficientes. Basicamente, o SO tenta descobrir, com base na história recente, quais páginas têm menos probabilidade de serem usadas no futuro próximo.



Simuladores de algoritmo de substituição de página

Uma discussão sobre algoritmos de substituição de página está fora do escopo deste livro. Uma técnica potencialmente eficaz se chama “usado menos recentemente” (LRU, do inglês *least recently used*), o mesmo algoritmo discutido no Capítulo 4 para a substituição de cache. Na prática, LRU é difícil de implementar para um esquema de paginação de memória virtual. Várias técnicas alternativas que buscam aproximar o desempenho do LRU são utilizadas; veja os detalhes no Apêndice F.

Com a paginação por demanda, não é necessário carregar um processo inteiro na memória principal. Esse fato tem uma consequência marcante: é possível que um processo seja maior do que toda a memória principal. Uma das restrições mais fundamentais na programação foi elevada. Se o programa sendo escrito for muito grande, o programador precisa criar maneiras de estruturar o programa em partes que possam ser carregadas uma de cada vez. Com a paginação por demanda, essa tarefa fica para o SO e o hardware. Já o programador está lidando com uma memória imensa, com o tamanho associado ao armazenamento em disco.

Como um processo só é executado na memória principal, essa memória é conhecida como **memória real ou física**. Mas um programador ou usuário percebe uma memória muito maior — aquela que está alocada no disco. Essa última, portanto, é denominada **memória virtual**. A memória virtual aumenta a eficiência da multiprogramação bastante eficaz, e alivia o usuário das restrições desnecessariamente apertadas da memória principal.

ESTRUTURA DA TABELA DE PÁGINA O mecanismo básico para a leitura de uma palavra da memória envolve a tradução de um endereço virtual, ou lógico, consistindo em número de página e deslocamento, para um endereço físico, consistindo em um número de frame e deslocamento, usando uma tabela de página. Como a tabela de página tem tamanho variável, dependendo do tamanho do processo, não podemos esperar mantê-la nos registradores. Em vez disso, ela precisa estar na memória principal para ser acessada. A Figura 8.16 sugere uma implementação de hardware desse esquema. Quando determinado processo está sendo executado, um registrador mantém o endereço inicial da tabela de página para esse processo. O número de página de um endereço virtual é usado para indexar essa tabela e pesquisar o número do frame correspondente. Isso é combinado com a parte de deslocamento do endereço virtual para produzir o endereço real desejado.

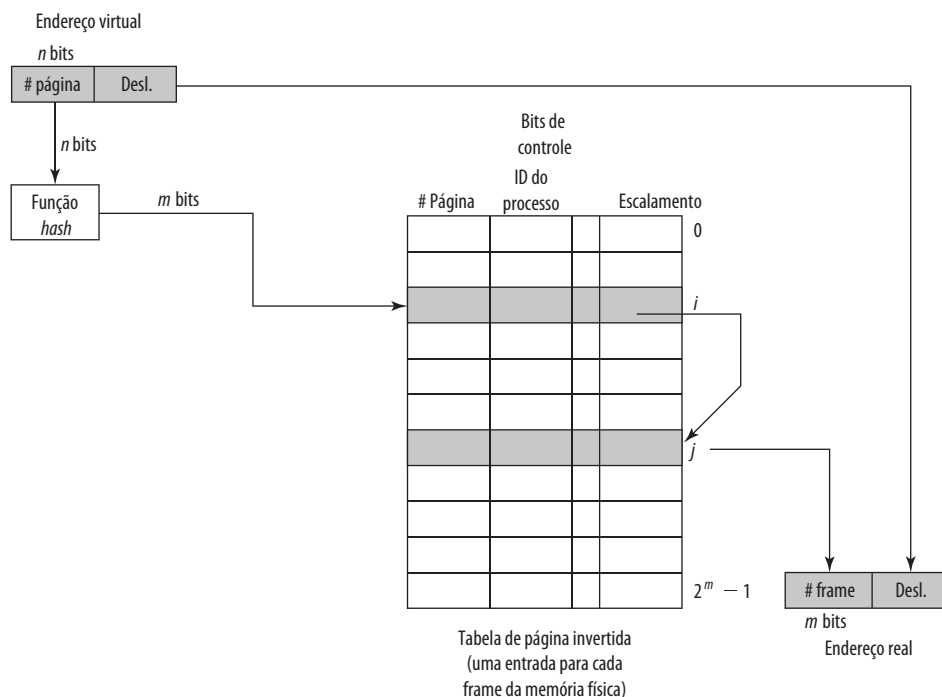
Na maioria dos sistemas, existe uma tabela de página por processo, mas cada processo pode ocupar grandes quantidades de memória virtual. Por exemplo, na arquitetura VAX, cada processo pode ter até $2^{31} = 2$ GBytes de memória virtual. Usando páginas de $2^9 = 512$ bytes, isso significa que até 2^{22} entradas da tabela de página são exigidas por processo. Logicamente, a quantidade de memória dedicada apenas a tabelas de página poderia ser inaceitavelmente alta. Para contornar esse problema, a maioria dos esquemas de memória virtual armazena as tabelas de página na memória virtual, e não na memória real. Isso significa que as tabelas de página são sujeitas a paginação, assim como as outras páginas. Quando um processo está sendo executado, pelo menos uma parte de sua tabela

de página precisa estar na memória principal, incluindo a entrada da tabela de página da página atualmente em execução. Alguns processadores utilizam um esquema de dois níveis para organizar grandes tabelas de página. Nesse esquema, existe um diretório de página, em que cada entrada aponta para uma tabela de página. Assim, se o tamanho do diretório de página for X , e se o tamanho máximo de uma tabela de página for Y , então um processo pode consistir em até $X \times Y$ páginas. Normalmente, o tamanho máximo de uma tabela de página é restrito a ser igual a uma página. Veremos um exemplo dessa técnica de dois níveis quando considerarmos o Pentium II mais adiante neste capítulo.

Uma técnica alternativa ao uso de tabelas de página de um ou dois níveis é o uso de uma estrutura de tabela de página invertida (Figura 8.17). As variações nessa técnica são usadas no PowerPC, UltraSPARC e a arquitetura IA-64. Uma implementação do Mach OS no RT-PC também usa essa técnica.

Nessa técnica, a parte do número de página de um endereço virtual é mapeada em um valor de *hash* usando uma função de *hashing* simples.² O valor de *hash* é um ponteiro para a tabela de página invertida, que contém as entradas da tabela de página. Existe uma entrada na tabela de página invertida para cada frame de página da memória real, em vez de uma por página virtual. Assim, uma proporção fixa da memória real é exigida para as tabelas, independentemente do número de processos ou páginas virtuais admitidas. Como mais de um endereço virtual pode ser mapeado na mesma entrada da tabela de *hash*, uma técnica de encadeamento é usada para gerenciar o *overflow*. A técnica de *hashing* resulta em cadeias que normalmente são curtas — entre uma e duas entradas. A estrutura da tabela de página é chamada invertida porque indexa as entradas da tabela de página por número de frame, em vez de número de página virtual.

Figura 8.17 Estrutura da tabela de página invertida



² Uma função de *hash* mapeia números na faixa de 0 a M para números na faixa de 0 a N , onde $M > N$. A saída da função de *hash* é usada como um índice para a tabela de *hash*. Como mais de uma entrada é mapeada para a mesma saída, é possível que um item de entrada seja mapeado para uma entrada da tabela de *hash* que já está ocupada. Nesse caso, o novo item precisa transbordar (*overflow*) para outro local da tabela de *hash*. Normalmente, o novo item é colocado no primeiro espaço vazio subsequente, e um ponteiro do local original é fornecido para encadear as entradas. Veja mais informações sobre funções de *hash* no Apêndice C.

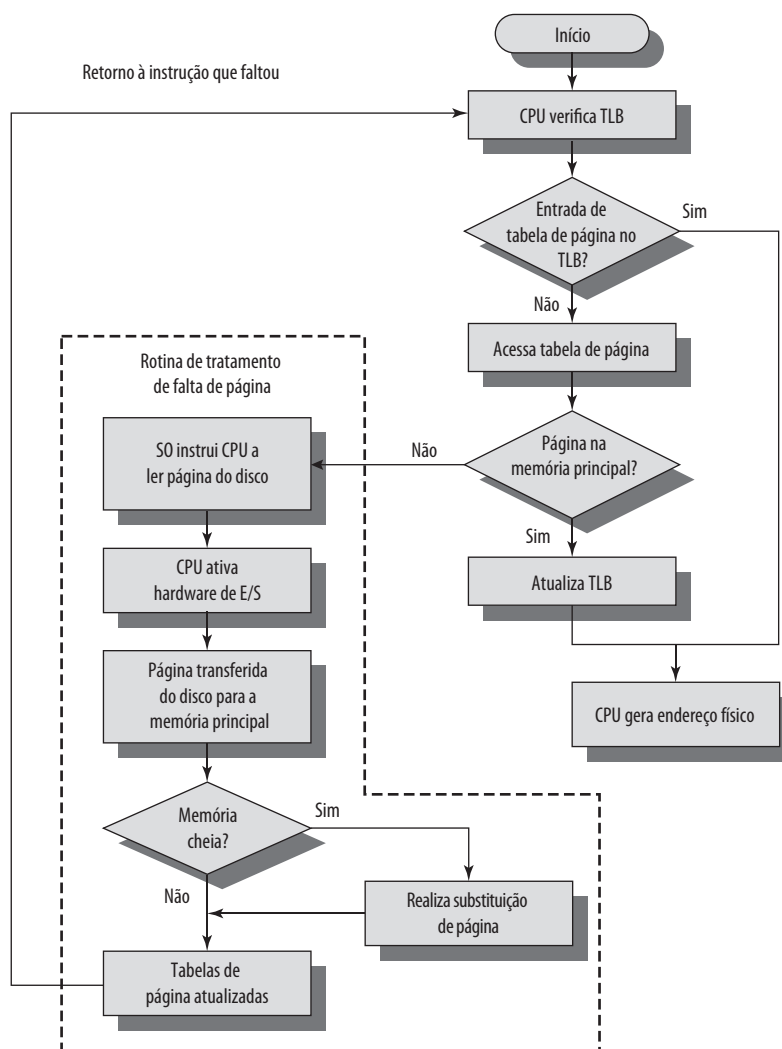


Translation lookaside buffer

Em princípio, então, cada referência de memória virtual pode causar dois acessos à memória física: um para buscar a entrada da tabela de página apropriada e um para buscar os dados desejados. Assim, um esquema direto de memória virtual teria o efeito de dobrar o tempo de acesso à memória. Para contornar esse problema, a maioria dos esquemas de memória virtual utiliza uma cache especial para entradas da tabela de página, normalmente chamada de *translation lookaside buffer* (TLB). Essa cache funciona da mesma maneira que uma cache de memória e contém as entradas da tabela de página que foram usadas recentemente. A Figura 8.18 é um fluxograma que mostra o uso do TLB. Pelo princípio da localidade, a maioria das referências à memória virtual será para locais nas páginas usadas recentemente. Portanto, a maioria das referências envolverá as entradas da tabela de página na cache. Os estudos do TLB do VAX mostraram que esse esquema pode melhorar o desempenho significativamente (Clark e Emer, 1985^a, Satyanarayanan e Bhandarkar, 1981^b).

Observe que o mecanismo de memória virtual precisa interagir com o sistema de cache (não a cache do TLB, mas a cache da memória principal). Isso é ilustrado na Figura 8.19. Um endereço virtual geralmente estará na forma de um número de página, deslocamento. Primeiro, o sistema de memória consulta o TLB para ver se a entrada da tabela de página correspondente está presente. Se estiver, o endereço real (físico) é gerado combinando o número

Figura 8.18 Operação da paginação e do *translation lookaside buffer* (TLB)



do *frame* com o deslocamento. Se não, a entrada é acessada a partir de uma tabela de página. Quando o endereço real for gerado, que está na forma de uma tag e um restante, a cache é consultada para ver se o bloco contendo essa palavra está presente (ver Figura 4.5). Nesse caso, ela retorna ao processador. Se não, a palavra é recuperada da memória principal.

O leitor deverá ser capaz de apreciar a complexidade do hardware do processador envolvida em uma única referência à memória. O endereço virtual é traduzido para um endereço real. Isso envolve referência a uma tabela de página, que pode estar no TLB, na memória principal ou no disco. A palavra referenciada pode estar na cache, na memória principal ou no disco. Nesse último caso, a página contendo a palavra precisa ser carregada para a memória principal e seu bloco carregado na cache. Além disso, a entrada da tabela de página para essa página precisa ser atualizada.

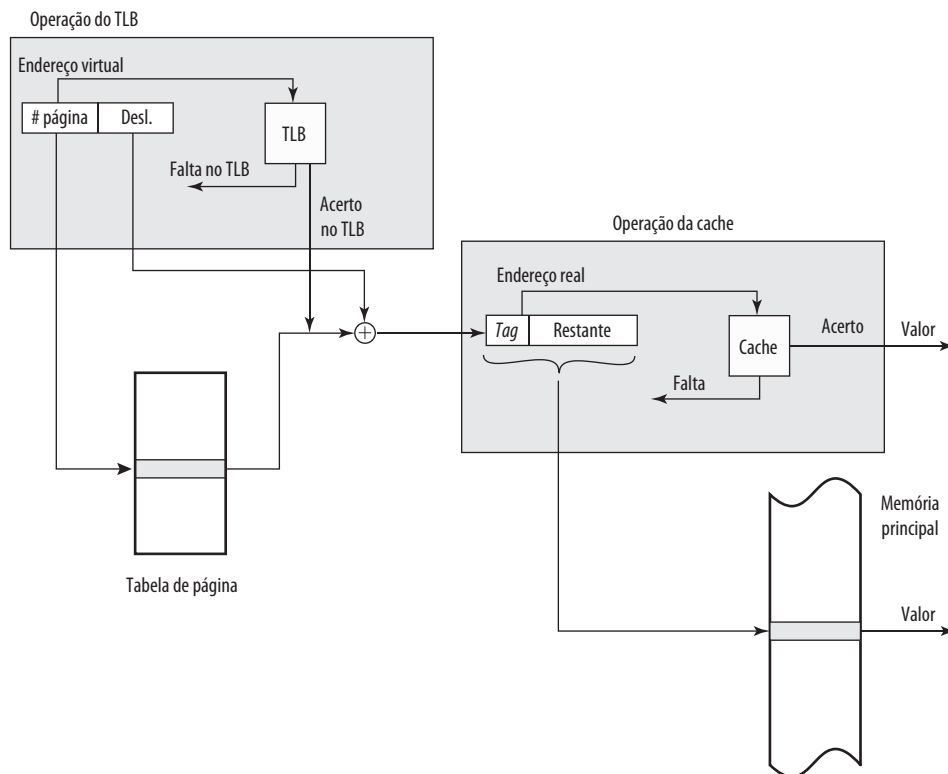


Segmentação

Existe outra maneira de subdividir a memória endereçável, conhecida como segmentação. Enquanto a paginação é invisível ao programador e serve para lhe oferecer um espaço de endereços maior, a segmentação normalmente é visível ao programador e é fornecida como uma conveniência para organizar programas e dados e como um meio de associar atributos de privilégio e proteção com instruções e dados.

A segmentação permite que o programador veja a memória como consistindo em múltiplos espaços ou segmentos de endereço. Os segmentos têm tamanho variável, realmente dinâmico. Normalmente, o programador ou o SO atribuirá programas e dados a diferentes segmentos. Pode haver uma série de segmentos de programa para diversos tipos de programas, além de uma série de segmentos de dados. Cada segmento pode ter direitos de acesso e uso atribuídos. As referências à memória consistem em uma forma de endereço (número de segmento, deslocamento).

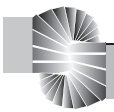
Figura 8.19 Translation lookaside buffer (TLB) e operação da cache



Essa organização tem diversas vantagens para o programador em relação a um espaço de endereços não segmentado:

1. Ela simplifica o tratamento de estruturas de dados que crescem. Se o programador não souber antes da hora o tamanho que determinada estrutura de dados terá, não é preciso adivinhar. A estrutura de dados pode receber seu próprio segmento, e o SO expandirá ou encolherá o segmento conforme a necessidade.
2. Ela permite que os programas sejam alterados e recompilados de modo independente, sem exigir que um conjunto inteiro de programas seja novamente link-editado e recarregado. Novamente, isso é feito usando segmentos múltiplos.
3. Ela permite o compartilhamento entre os processos. Um programador pode colocar um programa utilitário ou uma tabela de dados útil em um segmento que pode ser endereçado por outros processos.
4. Ela serve para proteção. Como um segmento pode ser construído para conter um conjunto bem definido de programas ou dados, o programador ou um administrador do sistema pode atribuir privilégios de acesso de uma maneira conveniente.

Essas vantagens não estão disponíveis com a paginação, que é invisível ao programador. Por outro lado, vimos que a paginação provê uma forma eficiente de gerenciamento de memória. Para combinar as vantagens de ambos, alguns sistemas são equipados com o hardware e o software do SO que permite o uso de ambos.



8.4 Gerenciamento de memória no Pentium

Desde a introdução da arquitetura de 32 bits, os microprocessadores evoluíram com sofisticados esquemas de gerenciamento de memória, que se baseiam nas lições aprendidas com os sistemas de média e grande escala. Em muitos casos, as versões dos microprocessadores são superiores aos seus antecedentes de sistemas maiores. Como os esquemas foram desenvolvidos pelo fornecedor de hardware do microprocessador e podem ser empregados com diversos sistemas operacionais, eles tendem a ser de uso bastante geral. Um exemplo representativo é o esquema usado no Pentium II. O hardware de gerenciamento de memória do Pentium II é basicamente o mesmo usado nos processadores Intel 80386 e 80486, com algumas melhorias.



Espaços de endereços

O Pentium II inclui hardware para segmentação e paginação. Os dois mecanismos podem ser desativados, permitindo que o usuário escolha a partir de quatro visões distintas da memória:

- **Memória não paginada não segmentada:** nesse caso, o endereço virtual é o mesmo que o endereço físico. Isso é útil, por exemplo, em aplicações de controlador de baixa complexidade e alto desempenho.
- **Memória paginada não segmentada:** aqui, a memória é vista como um espaço de endereço linear paginado. A proteção e o gerenciamento de memória são feitos por meio da paginação, o que é utilizado por alguns sistemas operacionais (por exemplo, Berkeley UNIX).
- **Memória não paginada segmentada:** aqui, a memória é vista como uma coleção de espaços de endereços lógicos. A vantagem dessa visão em relação à abordagem paginada é que ela proporciona proteção até o nível de um único byte, se for preciso. Além do mais, diferente da paginação, ela garante que a tabela de tradução necessária (a tabela de segmento) esteja no chip quando o segmento estiver na memória. Logo, a memória não paginada segmentada resulta em tempos de acesso previsíveis.
- **Memória paginada segmentada:** a segmentação é usada para definir partições de memória lógicas, sujeitas a controle de acesso, e a paginação é usada para gerenciar a alocação de memória dentro das partições. Sistemas operacionais como UNIX System V utilizam essa opção.



Segmentação

Quando a segmentação é usada, cada endereço virtual (chamado endereço lógico na documentação do Pentium II) consiste em uma referência de segmento de 16 bits e um offset de 32 bits. Dois bits da referência de segmento lidam com o mecanismo de proteção, deixando 14 bits para especificar um segmento em particular. Assim, com a memória não segmentada, a memória virtual do usuário é de $2^{32} = 4$ GBytes. Com a memória segmentada, o espaço total da memória virtual visto por um usuário é de $2^{46} = 64$ terabytes (TBytes). O espaço de endereço físico emprega um endereço de 32 bits para um máximo de 4 GBytes.

A quantidade de memória virtual pode realmente ser maior que 64 TBytes, porque a interpretação do processador de um endereço virtual depende de qual processo está atualmente ativo. O espaço de endereço virtual é dividido em duas partes. Metade do espaço de endereço virtual (8 K segmentos $\times$ 4 GBytes) é global, compartilhado por todos os processos; o restante é local e distinto para cada processo.

Associado a cada segmento estão duas formas de proteção: nível de privilégio e atributo de acesso. Existem quatro níveis de privilégio, do mais protegido (nível 0) ao menos protegido (nível 3). O nível de privilégio associado a um segmento de dados é sua “classificação”; o nível de privilégio associado a um segmento de programa é sua “autorização”. Um programa em execução só pode acessar segmentos de dados para os quais seu nível de autorização seja menor (mais privilegiado) que ou igual ao (mesmo privilégio) nível de privilégio do segmento de dados.

O hardware não dita como esses níveis de privilégio devem ser usados; isso depende do projeto e da implementação do SO. A intenção foi que o nível de privilégio 1 fosse usado para a maior parte do SO, e o nível 0 fosse usado para aquela pequena parte do SO dedicada ao gerenciamento de memória, proteção e controle de acesso. Isso deixa dois níveis para as aplicações. Em muitos sistemas, as aplicações residirão no nível 3, com o nível 2 não sendo usado. Os subsistemas de aplicação especializados, que precisam ser protegidos porque implementam seus próprios mecanismos de segurança, são bons candidatos para o nível 2. Alguns exemplos são sistemas de gerenciamento de banco de dados, sistemas de automação de escritórios e ambientes de softwares de engenharia.

Além de regular o acesso aos segmentos de dados, o mecanismo de privilégio limita o uso de certas instruções. Algumas instruções, como aquelas lidando com registradores de gerenciamento de memória, só podem ser executadas no nível 0. As instruções de E/S só podem ser executadas até um certo nível designado pelo SO; normalmente, esse será o nível 1.

O atributo de acesso de um segmento de dados especifica se os acessos de leitura/gravação ou apenas leitura são permitidos. Para segmentos de programa, o atributo de acesso especifica o acesso de leitura/execução ou somente leitura.

O mecanismo de tradução de endereço para a segmentação envolve o mapeamento de um endereço virtual no que é conhecido como endereço linear (Figura 8.20b). Um endereço virtual consiste no offset de 32 bits e um seletor de segmento de 16 bits (Figura 8.20a). O seletor de segmento consiste nos seguintes campos:

- **Indicador de tabela (TI, do inglês *table indicator*):** indica se a tabela de segmento global ou uma tabela de segmento local deve ser usada para tradução.
- **Número de segmento:** serve como um índice para a tabela de segmento.
- **Nível de privilégio do requisitante (RPL, do inglês *requestor privilege level*):** o nível de privilégio requisitado para esse acesso.

Cada entrada em uma tabela de segmento consiste em 64 bits, como mostra a Figura 8.20c. Os campos são definidos na Tabela 8.5.

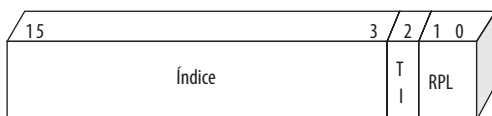
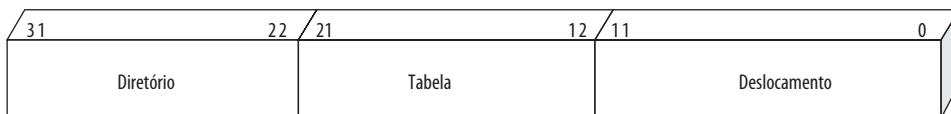
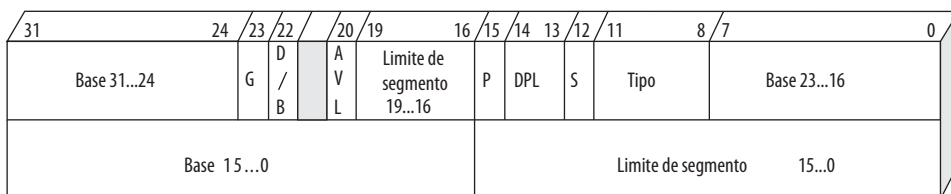


Paginação

A segmentação é um recurso opcional e pode ser desativado. Quando a segmentação está em uso, os endereços usados nos programas são endereços virtuais e são convertidos para endereços lineares, conforme descrevemos. Quando a segmentação não está em uso, os endereços lineares são usados nos programas. Nesse caso, a etapa a seguir é converter esse endereço linear em um endereço real de 32 bits.

Para entender a estrutura do endereço linear, é preciso saber que o mecanismo de paginação do Pentium II é, na realidade, uma operação de pesquisa de tabela em dois níveis. O primeiro nível é um diretório de página, que contém até 1.024 entradas. Isso divide o espaço de memória linear de 4 GBytes em 1.024 grupos de páginas, cada um com sua própria tabela de página, e cada um com 4 MBytes de extensão. Cada tabela de página contém até 1.024 entradas; cada entrada corresponde a uma única página de 4 Kbytes. O gerenciamento de memória tem a opção de usar um diretório de página para todos os processos, um diretório de página para cada processo, ou alguma combinação dos dois. O diretório de página para a tarefa atual está sempre na memória principal. As tabelas de página podem estar na memória virtual.

A Figura 8.20 mostra os formatos das entradas nos diretórios de página e tabelas de página, e os campos são definidos na Tabela 8.5. Observe que os mecanismos de controle de acesso podem ser fornecidos com base em uma página ou um grupo de páginas.

Figura 8.20 Formatos de gerenciamento de memória do PentiumTI = *table indicator* (indicador de tabela)RPL = *requestor privilege level* (nível de privilégio do requisitante)**(a) Seletor de segmento****(b) Endereço linear**AVL = *available* (disponível) para uso pelo software do sistema

Base = endereço do segmento de base

D/B = tamanho de operação padrão

DPL = tamanho de privilégio do descritor

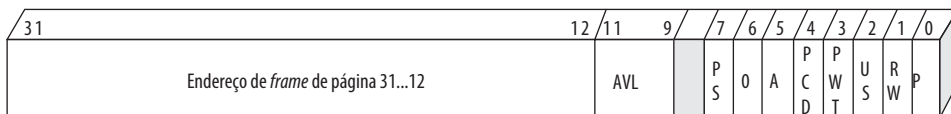
G = detalhamento

P = segmento presente

Tipo = tipo de segmento

S = tipo de descritor

□ = reservado

(c) Descritor de segmento (entrada da tabela de segmento)AVL = *available* (disponível) para uso do programador de sistemas

PS = tamanho da página

A = acessado

PCD = desabilitar cache de página

PWT = *write through*

US = usuário/supervisor

RW = *read/write* (leitura/gravação)

P = presente

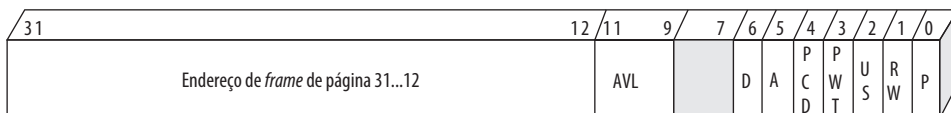
(d) Entrada de diretório de páginaD = *dirty* (modificado)**(e) Entrada da tabela de página**

Tabela 8.5 Parâmetros de gerenciamento de memória do Pentium II

Descritor de segmento (entrada da tabela de segmento)	
Base	
	Define o endereço inicial do segmento dentro do espaço de endereço linear de 4 GBytes.
Bit D/B	
	Em um segmento de código indica se os operandos e modos de endereçamento são de 16 ou 32 bits.
Descriptor privilege level (DPL)	
	Especifica o nível de privilégio do segmento referenciado por esse descritor de segmento.
Bit de detalhamento (G)	
	Indica se o campo Limite deve ser interpretado em unidades de um byte ou 4 KBytes.
Limite	
	Define o tamanho do segmento. O processador interpreta o campo de Limite de duas maneiras, dependendo do bit de detalhamento: em unidades de um byte, até um limite de tamanho de segmento de 1 MByte, ou em unidades de 4 KBytes, até um limite de tamanho de segmento de 4 GBytes.
Bit S	
	Determina se determinado segmento é um segmento do sistema ou um segmento de código ou dados.
Bit de segmento presente (P)	
	Usado para sistemas não paginados. Indica se o segmento está presente na memória principal. Para sistemas paginados, esse bit é sempre definido como 1.
Tipo	
	Distingue entre diversos tipos de segmentos e indica os atributos de acesso.
Entrada de diretório de página e entrada de tabela de página	
Bit acessado (A)	
	Esse bit é definido como 1 pelo processador nos dois níveis de tabelas de página quando ocorre uma operação de leitura ou gravação na página correspondente.
Bit de modificação (D)	
	Esse bit é definido como 1 pelo processador quando ocorre uma operação de gravação na página correspondente.
Endereço de frame de página	
	Oferece o endereço físico da página na memória se o bit presente estiver marcado. Como os frames de página são alinhados em limites de 4 K, os 12 bits inferiores são 0, e somente os 20 bits superiores são inclusos na entrada. Em um diretório de página, o endereço é o de uma tabela de página.
Bit desabilitar cache de página (PCD, do inglês <i>Page cache disable</i>)	
	Indica se os dados da página podem ser colocados em cache.
Bit de tamanho de página (PS, do inglês <i>page size</i>)	
	Indica se o tamanho de página é de 4 KBytes ou 4 MBytes.
Bit de write-through de página (PWT — <i>Page Write Through</i>)	
	Indica se a política de cache <i>write-through</i> ou <i>write-back</i> será usada para os dados na página correspondente.
Bit de presente (P)	
	Indica se a tabela de página ou página está presente na memória principal.
Bit de leitura/escrita (RW — <i>read/write</i>)	
	Para páginas em nível de usuário, indica se a página é de acesso apenas de leitura ou acesso de leitura/gravação para programas em nível de usuário.
Bit de usuário/supervisor (US)	
	Indica se a página está disponível apenas para o sistema operacional (nível supervisor) ou se está disponível para o sistema operacional e as aplicações (nível de usuário).

O Pentium II também utiliza um TLB. O buffer pode manter 32 entradas de tabela de página. Toda vez que o diretório de página é alterado, o buffer é apagado.

A Figura 8.21 ilustra a combinação de mecanismos de segmentação e paginação. Por clareza, o TLB e os mecanismos de cache de memória não aparecem.

Finalmente, o Pentium II inclui uma nova extensão não encontrada no 80386 ou 80486, permite dois tamanhos de página. Se o bit PSE (extensão de tamanho de página) no registrador de controle 4 estiver definido como 1, então a unidade de paginação permite que o programador do SO defina uma página como 4 KByte ou 4 MByte de tamanho.

Quando páginas de 4 MBytes são usadas, existe apenas um nível de pesquisa de tabela para páginas. Quando o hardware acessa o diretório de página, a entrada do diretório de página (Figura 8.20d) tem o bit PS definido como 1. Nesse caso, os bits de 9 a 21 são ignorados e os bits de 22 a 31 definem o endereço de base para uma página de 4 MBytes na memória. Assim, existe uma única tabela de página.

O uso de páginas de 4 MBytes reduz os requisitos de armazenamento do gerenciamento de memória para grandes memórias principais. Com páginas de 4 KBytes, uma memória principal completa de 4 GBytes requer cerca de 4 MBytes de memória só para as tabelas de página. Com páginas de 4 MBytes, uma única tabela, com 4 KBytes de extensão, é suficiente para o gerenciamento de memória de página.



8.5 Gerenciamento de memória no ARM

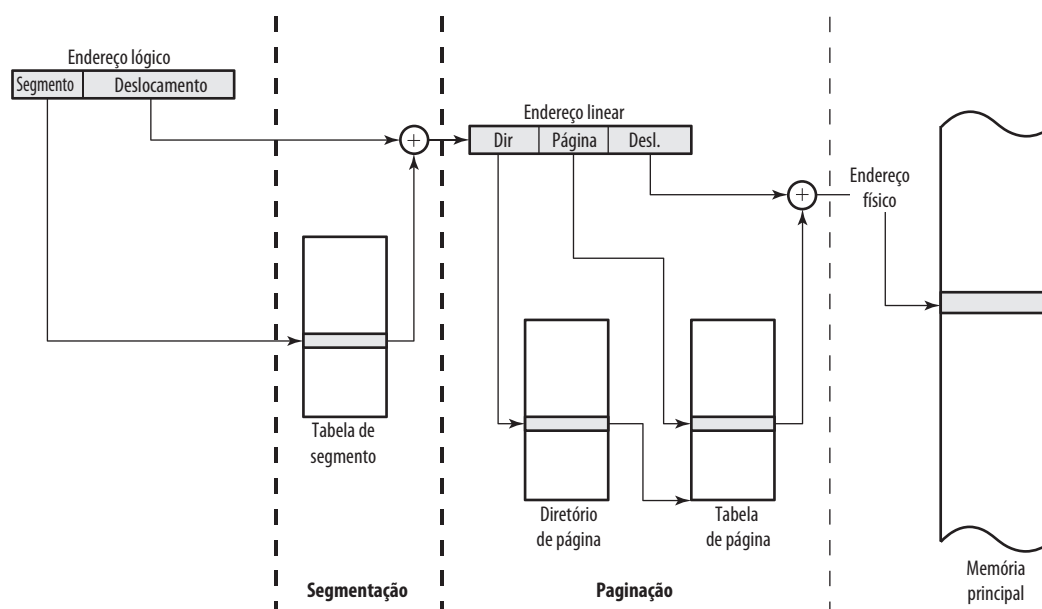
O ARM oferece uma arquitetura versátil de sistema de memória virtual, que pode ser ajustada às necessidades do projetista de sistema embarcado.



Organização do sistema de memória

A Figura 8.22 oferece uma visão geral do hardware de gerenciamento de memória no ARM para a memória virtual. O hardware de tradução da memória virtual usa um ou dois níveis de tabelas para tradução de endereços virtuais para físicos, conforme explicamos mais adiante. O TLB é uma cache de entradas recentes tabela de página. Se uma entrada estiver disponível no TLB, então o TLB envia diretamente um endereço físico para a memória prin-

Figura 8.21 Mecanismos de tradução de endereço de memória no Pentium



cial, para uma operação de leitura ou escrita. Conforme explicamos no Capítulo 4, os dados são trocados entre o processador e a memória principal por meio da cache. Se for usada uma organização lógica de cache (Figura 4.7a), então o ARM fornece esse endereço diretamente à cache, além de fornecê-lo ao TLB quando houver uma falta de cache. Se for usada uma organização física de cache (Figura 4.7b), então o TLB precisa fornecer o endereço físico para a cache.

As entradas nas tabelas de tradução também incluem bits de controle de acesso, que especificam se determinado processo pode acessar determinada parte da memória. Se o acesso for negado, o hardware de controle de acesso fornece ao processador ARM um sinal de abortar.



Tradução de endereço da memória virtual

O ARM admite acesso à memória com base em seções ou páginas:

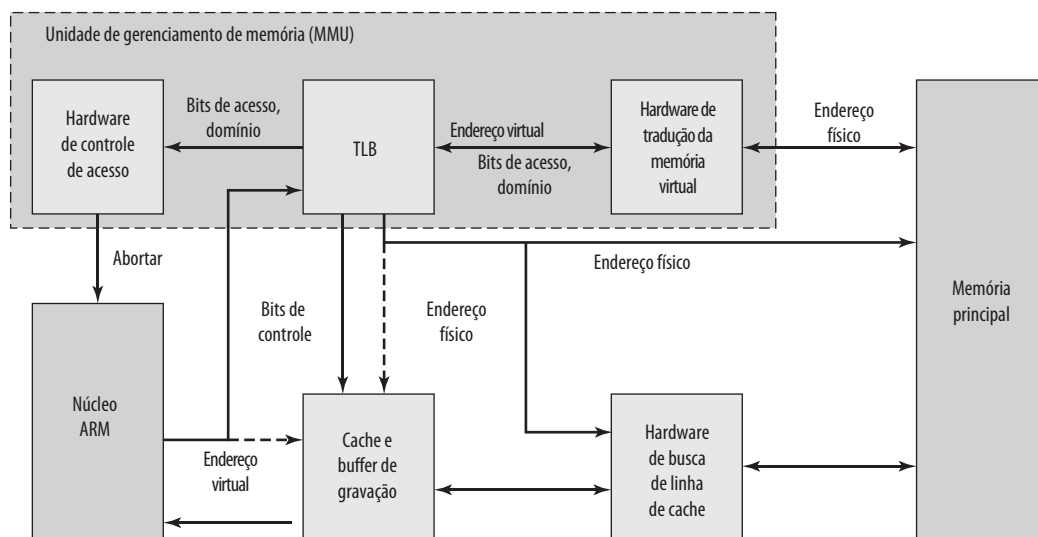
- **Superseções (opcional):** consistem em blocos de 16 MB de memória principal.
- **Seções:** consistem em blocos de 1 MB de memória principal.
- **Páginas grandes:** consistem em blocos de 64 KB de memória principal.
- **Páginas pequenas:** consistem em blocos de 4 KB de memória principal.

Seções e superseções são usadas para permitir o mapeamento de uma grande região da memória enquanto utilizam apenas uma única entrada no TLB. Outros mecanismos de controle de acesso são estendidos dentro das páginas pequenas a subpáginas de 1 KB, e dentro das páginas grandes, a subpáginas de 16 KB. A tabela de tradução mantida na memória principal tem dois níveis:

- **Tabela de primeiro nível:** mantém traduções de seção e superseção, e ponteiros para tabelas de segundo nível.
- **Tabelas de segundo nível:** mantêm traduções de página grandes e pequenas.

A unidade de gerenciamento de memória (MMU) traduz os endereços virtuais gerados pelo processador em endereços físicos para acessar a memória principal, e também deriva e verifica a permissão de acesso. As traduções ocorrem como resultado de uma falta da TLB, e começam com uma busca de primeiro nível. Um acesso mapeado por seção só requer uma busca de primeiro nível, enquanto um acesso mapeado por página também requer uma busca de segundo nível.

Figura 8.22 Visão geral do sistema de memória do ARM



A Figura 8.23 mostra o processo de tradução de endereço de dois níveis para páginas pequenas. Existe uma única tabela de página de nível 1 (L1) com 4 K entradas de 32 bits. Cada entrada L1 aponta para uma tabela de página de nível 2 (L2) com 255 entradas de 32 bits. Cada entrada de L2 aponta para uma página de 4 KB na memória principal. O endereço virtual de 32 bits é interpretado da seguinte forma: os 12 bits mais significativos são um índice para a tabela de página L1. Os próximos 8 bits são um índice para a tabela de página L2 relevante. Os 12 bits menos significativos indexam um byte na página relevante da memória principal.

Um procedimento semelhante de pesquisa de duas páginas é utilizado para páginas grandes. Para seções e superseção, apenas a pesquisa da tabela de página L1 é necessária.



Formatos de gerenciamento de memória

Para entender melhor o esquema de gerenciamento de memória do ARM, consideramos os principais formatos, como mostra a Figura 8.24. Os bits de controle mostrados nessa figura são definidos na Tabela 8.6.

Para a tabela L1, cada entrada descreve como sua faixa de endereços virtuais de 1 MB associados é mapeada. Cada entrada tem um de quatro formatos alternativos:

Bits [1:0] = 00: os endereços virtuais associados são não mapeados, e as tentativas de acessá-los geram uma falha de tradução.

Bits [1:0] = 01: a entrada dá o endereço físico de uma tabela de página L2, que especifica como o intervalo de endereço virtual associado é mapeado.

Bits [1:0] = 01 e bit 19 = 0: a entrada é um descritor de seção para seus endereços virtuais associados.

Bits [1:0] = 01 e bit 19 = 1: a entrada é um descritor de superseção para seus endereços virtuais associados.

As entradas com bits [1:0] = 11 são reservadas.

Figura 8.23 Tradução de endereço de memória virtual do ARM para páginas pequenas

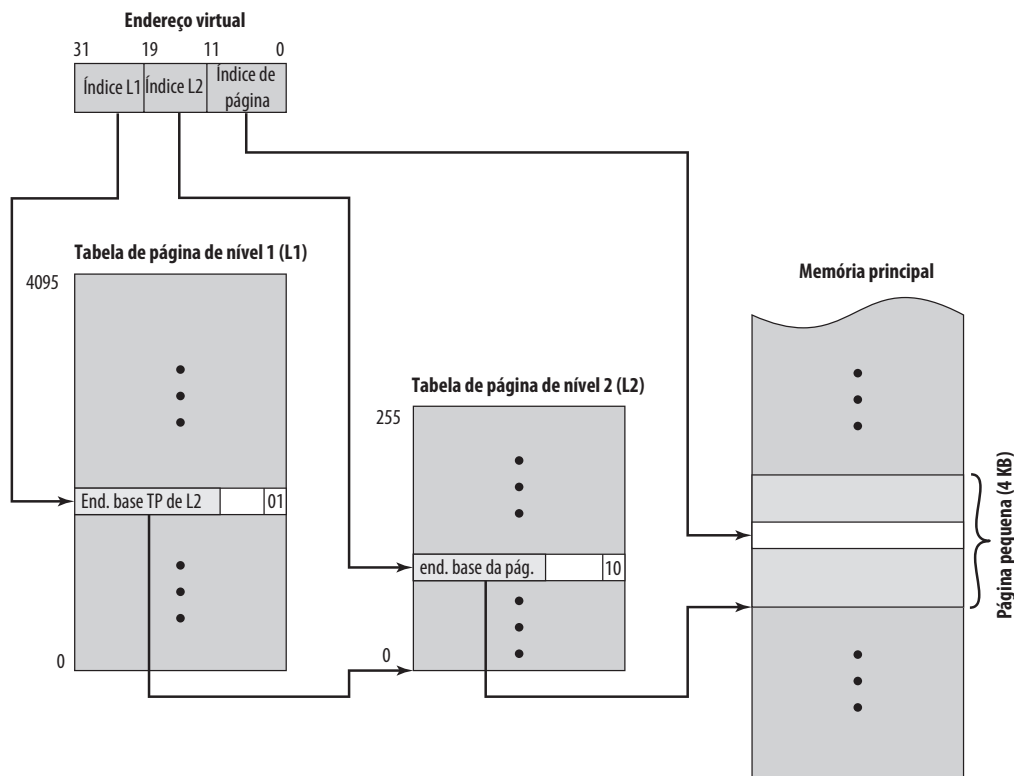
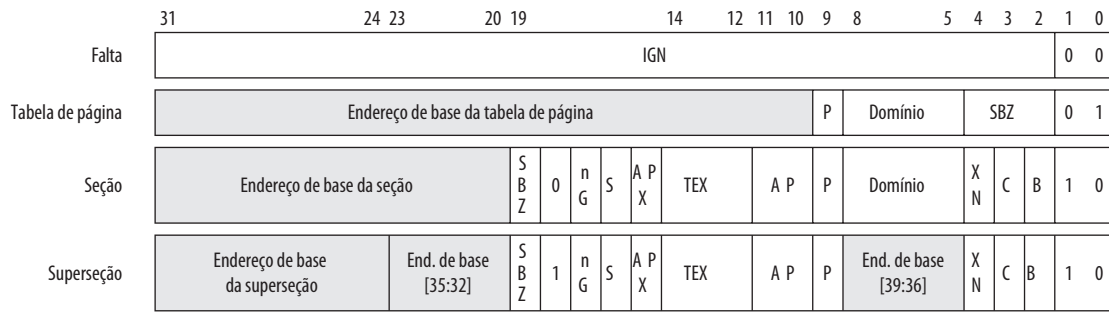
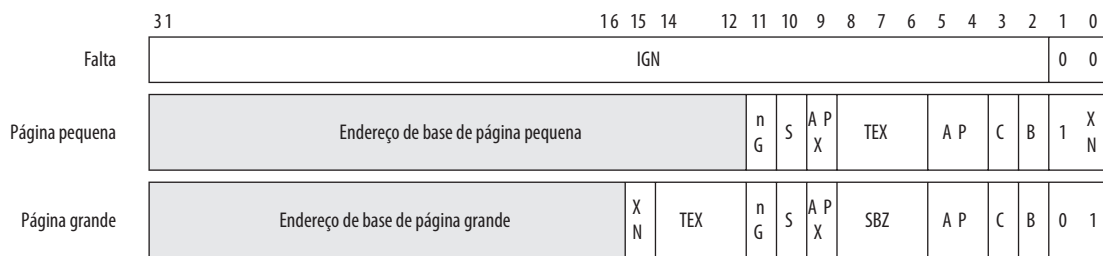
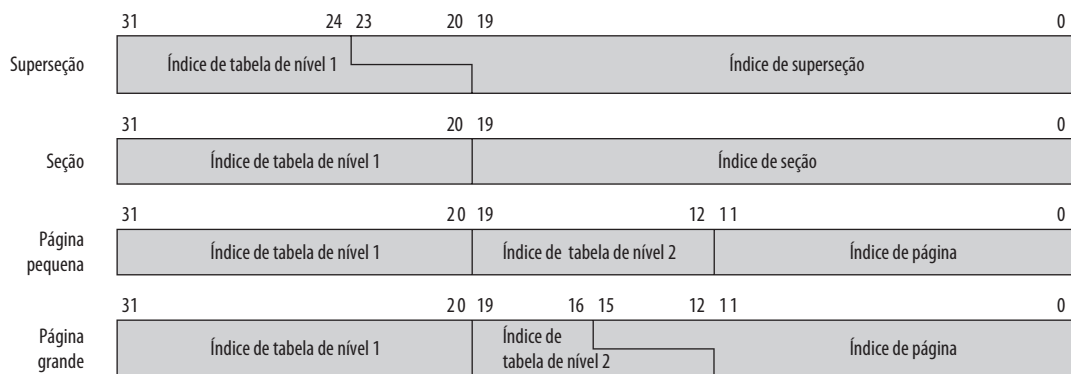


Figura 8.24 Formatos de gerenciamento de memória do ARMv6**(a) Formatos alternativos de descritor de primeiro nível****(b) Formatos alternativos de descritor de segundo nível****(c) Formatos de endereço de memória virtual**

Para a memória estruturada em páginas, é preciso haver um acesso à tabela de página em dois níveis. Os bits [31:10] da entrada de página L1 contêm um ponteiro para uma tabela de página L1. Para páginas pequenas, a entrada L2 contém um ponteiro de 20 bits para o endereço de base de uma página de 4 KB na memória principal.

Para páginas grandes, a estrutura é mais complexa. Assim como os endereços virtuais para páginas pequenas, um endereço virtual para uma estrutura de página grande inclui um índice de 12 bits para a tabela de nível 1 e um índice de 8 bits para a tabela L2. Para as páginas grandes de 64 KB, a parte de índice de página do endereço virtual precisa ser de 16 bits. Para acomodar todos esses bits em um formato de 32 bits, existe uma sobreposição de 4 bits entre o campo de índice de página e o campo de índice de tabela L2. O ARM acomoda essa sobreposição exigindo que cada entrada da tabela de página em uma tabela de página L2, que aceita páginas grandes, seja replicada 16 vezes. Com efeito, o tamanho da tabela de página L2 é reduzido de 256 entradas para 16 entradas, se todas as

Tabela 8.6 Parâmetros de gerenciamento de memória do ARM

Permissão de acesso (AP — <i>access permission</i>), Extensão de permissão de acesso (APX — <i>access permission extension</i>)
Esses bits controlam o acesso à região de memória correspondente. Se um acesso for feito a uma área da memória sem as permissões exigidas, uma falta de permissão é levantada.
Bit bufferable (B)
Determina, com os bits TEX, como o buffer de gravação é usado para a memória cacheável.
Bit cacheable (C)
Determina se essa região da memória pode ser mapeada pela cache.
Domínio
Coleção de regiões da memória. O controle de acesso pode ser aplicado com base no domínio.
não Global (nG)
Determina se a tradução deve ser marcada como global (0) ou específica ao processo (1).
Compartilhado (S — <i>shared</i>)
Determina se a tradução é para a memória não compartilhada (0) ou compartilhada (1).
SBZ
<i>Should Be Zero</i> (deverá ser zero).
Extensão de tipo (TEX — <i>type extension</i>)
Esses bits, juntamente com os bits B e C, controlam os acessos às caches, como o buffer de gravação é usado e se a região da memória é compartilhável e, portanto, deve ser mantida coerente.
Executar Nunca (XN)
Determina se a região é executável (0) ou não executável (1).

entradas referirem-se a páginas grandes. Porém, uma determinada página L2 pode atender a uma mistura de páginas grandes e pequenas, daí a necessidade de replicação para as entradas de página grande.

Para a memória estruturada em seções ou superseções, um acesso à tabela de página de um nível é necessário. Para seções, os bits [31:20] da entrada L1 contêm um ponteiro de 12 bits para a base da seção de 1 MB na memória principal.

Para superseções, os bits [31:24] da entrada L1 contêm um ponteiro de 8 bits para a base da seção de 16 MB na memória principal. Assim como as páginas grandes, uma replicação da entrada da tabela de página é necessária. No caso de superseções, a parte do índice da tabela L1 do endereço virtual sobrepõe por 4 bits com a parte de índice da superseção do endereço virtual. Portanto, 16 entradas da tabela de página L1 idênticas são necessárias.

O intervalo de espaço de endereço físico pode ser expandido por até oito bits de endereço adicionais (bits [23:20] e [8:5]). O número de bits adicionais depende da implementação. Esses bits adicionais podem ser interpretados como estendendo o tamanho da memória física por até $2^8 = 256$. Assim, a memória física de fato pode ser de até 256 vezes o tamanho do espaço de memória disponível a cada processo individual.



Controle de acesso

Os bits de controle de acesso AP em cada entrada de tabela controlam o acesso a uma região da memória por determinado processo. Uma região da memória pode ser designada como sem acesso, apenas de leitura ou leitura-

-escrita. Além disso, a região pode ser designada como acesso privilegiado apenas, reservado para uso pelo SO e não pelas aplicações.

O ARM também emprega o conceito de um domínio, que é uma coleção de seções e/ou páginas que possuem permissões de acesso particulares. A arquitetura ARM admite 16 domínios. O recurso de domínio permite que múltiplos processos usem as mesmas tabelas de tradução enquanto mantêm algumas proteções umas das outras.

Cada entrada de tabela de página e entrada de TLB contém um campo que especifica em qual domínio a entrada se encontra. Um campo de 2 bits no *Domain Access Control Register* controla o acesso a cada domínio. Cada campo permite que o acesso a um domínio inteiro seja ativado ou desativado muito rapidamente, de modo que áreas inteiras da memória possam entrar e sair da memória virtual de modo bastante eficiente. Dois tipos de acesso de domínio são aceitos:

- **Clientes:** usuários de domínios (executam programas e acessam dados) que precisam observar as permissões de acesso das seções individuais e/ou páginas que compõem esse domínio.
- **Gerentes:** controlam o comportamento do domínio (as seções e páginas atuais no domínio, e o acesso do domínio), e contornam as permissões de acesso para entradas de tabela nesse domínio.

Um programa pode ser um cliente de alguns domínios, e um gerente de alguns outros, e não ter acesso aos domínios restantes. Isso permite uma proteção de memória bastante flexível para programas que acessam diferentes recursos da memória.



8.6 Leitura recomendada e sites Web

Stallings (2009)^c aborda os tópicos deste capítulo com detalhes.



Sites Web recomendados

Operating System Resource Center: uma coleção útil de documentos e artigos sobre diversos tópicos de sistema operacional.

ACM Special Interest Group on Operating Systems: informações sobre publicações e conferências do SIGOPS.

IEEE Technical Committee on Operating Systems and Applications: inclui um boletim on-line e links para outros sites.

Principais termos, perguntas de revisão e problemas

Principais termos

Sistema em lote	Multitarefa	Monitor residente
Paginação por demanda	Núcleo	Segmentação
Sistemas operacionais interativos	Sistema operacional (SO)	Escalonamento de curto prazo
Interrupção	Paginação	Swapping
<i>Job control language</i> (JCL)	Tabela de página	Thrashing
Kernel	Particionamento	Sistema de tempo compartilhado
Endereço lógico	Endereço físico	<i>Translation lookaside buffer</i> (TLB)
Escalonamento de longo prazo	Instrução privilegiada	Programa utilitário
Escalonamento de médio prazo	Processo	Memória virtual
Gerenciamento de memória	Bloco de controle de processo	
Proteção de memória	Estado do processo	
Multiprogramação	Memória real	

Perguntas de revisão

- 8.1 O que é um sistema operacional?
- 8.2 Liste e defina resumidamente os principais serviços fornecidos por um sistema operacional.
- 8.3 Liste e defina resumidamente os principais tipos de escalonamento do sistema operacional.
- 8.4 Qual é a diferença entre um processo e um programa?
- 8.5 Qual é a finalidade do swapping?
- 8.6 Se um processo pode ser atribuído dinamicamente a diferentes locais na memória principal, qual é sua implicação para o mecanismo de endereçamento?
- 8.7 É necessário que todas as páginas de um processo estejam na memória enquanto o processo está sendo executado?
- 8.8 As páginas de um processo na memória principal precisam ser contíguas?
- 8.9 É necessário que as páginas de um processo na memória principal estejam em ordem sequencial?
- 8.10 Qual é a finalidade do *translation lookaside buffer* (TLB)?

Problemas

- 8.1 Suponha que tenhamos um computador multiprogramado em que cada job tenha características idênticas. Em um período de computação, T , para uma tarefa, metade do tempo é gasto na E/S e a outra metade na atividade do processador. Cada job é executado por um total de N períodos. Suponha que uma prioridade round-robin simples seja usada, e que as operações de E/S possam se sobrepor com a operação do processador. Defina as seguintes quantidades:

- Tempo de *turnaround* = tempo real para completar um job.
- *Throughput* = número médio de jobs completados por período T .
- Utilização do processador = percentagem de tempo que o processador está ativo (não esperando).

Calcule essas quantidades para um, dois e quatro jobs simultâneos, supondo que o período T seja distribuído em cada uma das seguintes maneiras:

- a. E/S primeira metade, processador segunda metade.
 - b. E/S primeira e quarta partes, processador segunda e terceira partes.
- 8.2 Um programa voltado para E/S é tal que, se executado sozinho, gastaria mais tempo esperando pela E/S do que usando o processador. Um programa voltado para o processador é o oposto. Suponha que o algoritmo de escalonamento a curto prazo favoreça aqueles programas que usaram pouco tempo de processador recentemente. Explique por que esse algoritmo favorece os programas voltados para a E/S e ainda assim não nega permanentemente o tempo do processador para os programas voltados para o processador.
- 8.3 Um programa calcula as somas de linhas

$$C_i = \sum_{j=1}^n a_{ij}$$

de um array A de 100 por 100 elementos. Suponha que o computador use a paginação por demanda com um tamanho de página de 1.000 palavras, e que a quantidade de memória principal alocada para dados seja de cinco *frames* de página. Existe alguma diferença na taxa de falta de página se A fosse armazenado na memória virtual, tendo por parâmetro linhas ou colunas? Explique.

- 8.4 Considere um esquema de particionamento com partições de mesmo tamanho de 2^{16} bytes e um tamanho de memória principal total de 2^{24} bytes. Uma tabela de processo é mantida, que inclui um ponteiro para uma partição para cada processo residente. Quantos bits são exigidos para o ponteiro?
- 8.5 Considere um esquema de particionamento dinâmico. Mostre que, na média, a memória contém uma quantidade de buracos que é a metade do número de segmentos.
- 8.6 Suponha que a tabela de página para o processo atualmente em execução no processador se pareça com a seguinte. Todos os números são decimais, tudo é numerado a partir de zero e todos os endereços são endereços de byte da memória. O tamanho de página é de 1.024 bytes.

Número da página virtual	Bit válido	Bit de referência	Bit de modificação	Número do frame de página
0	1	1	0	4
1	1	1	1	7
2	0	0	0	—
3	1	0	0	2
4	0	0	0	—
5	1	0	1	0

- a. Descreva exatamente como, em geral, um endereço virtual gerado pela CPU é traduzido para um endereço físico da memória principal.
- b. A que endereço físico, se houver algum, cada um dos seguintes endereços virtuais corresponde? (Não tente tratar de quaisquer faltas de página, se houver.)
 - (i) 1.052
 - (ii) 2.221
 - (iii) 5.499

8.7 Dê motivos para o tamanho de página em um sistema de memória virtual não ser nem muito pequeno nem muito grande.

8.8 Um processo referencia cinco páginas, A, B, C, D e E, na seguinte ordem:

A; B; C; D; A; B; E; A; B; C; D; E

Suponha que o algoritmo de substituição seja “primeiro a entrar, primeiro a sair” (FIFO) e encontre o número de transferências de página durante essa sequência de referências, começando com uma memória principal vazia com três *frames* de página. Repita para quatro *frames* de página.

8.9 A sequência de números de página virtual a seguir é encontrada no curso de execução em um computador com memória virtual:

3 4 2 6 4 7 1 3 2 6 3 5 1 2 3

Suponha que seja adotada uma política de substituição de página usada menos recentemente (LRU). Desenhe um gráfico da razão de acerto de página (fração de referências de página em que a página se encontra na memória principal) como uma função da capacidade da página de memória principal n para $1 \leq n \leq 8$. Suponha que a memória principal esteja inicialmente vazia.

8.10 No computador VAX, as tabelas de página do usuário estão localizadas nos endereços virtuais no espaço do sistema. Qual é a vantagem de ter tabelas de página do usuário na memória virtual, e não na memória principal? Qual é a desvantagem?

8.11 Suponha que a instrução de programa

para ($i = 1; i \leq n; i++$)

$a[i] = b[i] + c[i];$

seja executada em uma memória com tamanho de página de 1.000 palavras. Considere $n = 1.000$. Usando uma máquina que possui uma faixa completa de instruções registrador-para-registrador e emprega registradores de índice, escreva um programa hipotético para implementar a instrução indicada. Depois, mostre a sequência de referências de página durante a execução.

8.12 A arquitetura do IBM System/370 utiliza uma estrutura de memória de dois níveis e refere-se aos dois níveis como segmentos e páginas, embora a técnica de segmentação não tenha muitos dos recursos descritos anteriormente neste capítulo. Para a arquitetura 370 básica, o tamanho de página pode ser 2 Kbytes ou 4 Kbytes, e o tamanho do segmento é fixo em 64 Kbytes ou 1 MByte. Para as arquiteturas 370/XA e 370/ESA, o tamanho de página é de 4 Kbytes e o tamanho de segmento é de 1 MByte. Que vantagens da segmentação esse esquema não possui? Qual é o benefício da segmentação para o 370?

8.13 Considere um sistema de computação com segmentação e paginação. Quando um segmento está na memória, algumas palavras são desperdiçadas na última página. Além disso, para um tamanho de segmento s e tamanho de página p , existem s/p entradas de tabela de página. Quanto menor o tamanho da página, menor o desperdício na última página do segmento, porém maior a tabela de página. Que tamanho de página minimiza o *overhead* total?

8.14 Um computador tem uma cache, uma memória principal e um disco usados para memória virtual. Se uma palavra referenciada estiver na cache, 20 ns são necessários para acessá-la. Se estiver na memória principal, mas não na cache, 60 ns são necessários para carregá-la para a cache, e depois a referência é iniciada novamente. Se a palavra não estiver na memória principal, 12 ms são necessários para apanhar a

palavra do disco, seguidos por 60 ns para copiá-la para a cache e depois a referência é iniciada novamente. A razão de acerto de cache é 0,9 e a razão de acerto da memória principal é 0,6. Qual é o tempo médio em ns necessário para acessar uma palavra referenciada nesse sistema?

- 8.15** Considere que uma tarefa é dividida em quatro segmentos de mesmo tamanho e que o sistema monte uma tabela de descritor de página com oito entradas para cada segmento. Assim, o sistema tem uma combinação de segmentação e paginação. Suponha também que o tamanho de página seja de 2 KBytes.
- Qual é o tamanho máximo de cada segmento?
 - Qual é o espaço de endereço lógico para a tarefa?
 - Suponha que um elemento no local físico 00021ABC seja acessado por essa tarefa. Qual é o formato do endereço lógico que a tarefa gera para ela? Qual é o espaço de endereço físico máximo para o sistema?
- 8.16** Considere um microprocessador capaz de acessar até 2^{32} bytes de memória principal física. Ele implementa um espaço de endereço lógico segmentado de tamanho máximo 2^{31} bytes. Cada instrução contém o endereço inteiro em duas partes. Unidades de gerenciamento de memória (MMU) externas são usadas, cujo esquema de endereçamento atribui blocos contíguos de memória física de tamanho fixo de 2^{22} bytes aos segmentos. O endereço físico inicial de um segmento sempre é divisível por 1.024. Mostre a ligação detalhada do mecanismo de mapeamento externo que converte endereços lógicos em endereços físicos usando o número apropriado de MMU, e mostre a estrutura interna detalhada de uma MMU (supondo que cada MMU contenha uma cache de descritor de segmento mapeado diretamente com 128 entradas) e como cada MMU é selecionada.
- 8.17** Considere um espaço de endereço lógico paginado (composto de 32 páginas de 2 KBytes cada) mapeado em um espaço de memória física de 1 MByte.
- Qual é o formato do endereço lógico do processador?
 - Qual é a extensão e a largura da tabela de página (desconsiderando os bits de “direitos de acesso”)?
 - Qual é o efeito sobre a tabela de página se o espaço físico de memória for reduzido pela metade?
- 8.18** No sistema operacional de mainframe do IBM OS/390, um dos principais módulos no kernel é o *system resource manager* (SRM). Esse módulo é responsável pela alocação de recursos entre os espaços de endereço (processos). O SRM dá ao OS/390 um grau de sofisticação exclusivo entre os sistemas operacionais. Nenhum outro SO de mainframe, e certamente nenhum outro tipo de SO, pode corresponder às funções realizadas pelo SRM. O conceito de recurso inclui processador, memória real e canais de E/S. O SRM acumula estatísticas pertencentes à utilização do processador, canal e diversas estruturas de dados básicas. Sua finalidade é oferecer desempenho ideal com base no monitoramento e na análise de desempenho. A instalação destaca diversos objetivos de desempenho, e estes servem como guia para o SRM, que modifica dinamicamente as características de instalação e desempenho do job com base na utilização do sistema. Por sua vez, o SRM oferece relatórios que permitem que o operador treinado refine a configuração e as definições de parâmetros para melhorar o serviço ao usuário.
- Este problema trata de um exemplo da atividade do SRM. A memória real é dividida em blocos de mesmo tamanho, chamados *frames*, dos quais pode haver muitos milhares. Cada *frame* pode manter um bloco de memória virtual conhecido como página. O SRM recebe o controle aproximadamente 20 vezes por segundo e inspeciona todo e qualquer *frame* de página. Se a página não tiver sido referenciada ou alterada, um contador é incrementado em 1. Com o passar do tempo, o SRM calcula a média desses números para determinar o número médio de segundos que um *frame* de página no sistema fica sem ser tocado. Qual poderia ser a finalidade disso, e que ação o SRM poderia tomar?
- 8.19** Para cada um dos formatos de endereço virtual do ARM mostrados na Figura 8.24, mostre o formato do endereço físico.
- 8.20** Desenhe uma figura semelhante à Figura 8.23 para a tradução da memória virtual do ARM quando a memória principal é dividida em seções.

Referências

- CLARK, D. e EMER, J. “Performance of the VAX-11/780 translation buffer: simulation and Measurement”. *ACM Transactions on Computer Systems*, fev. 1985.
- SATYANARAYANAN, M. e BHANDARKAR, D. “Design trade-offs in VAX-11 translation buffer organization”. *Computer*, dez. 1981.
- STALLINGS, W. *Operating systems, internals and design principles*, 6 ed. Upper Saddle River, NJ: Prentice Hall, 2009.